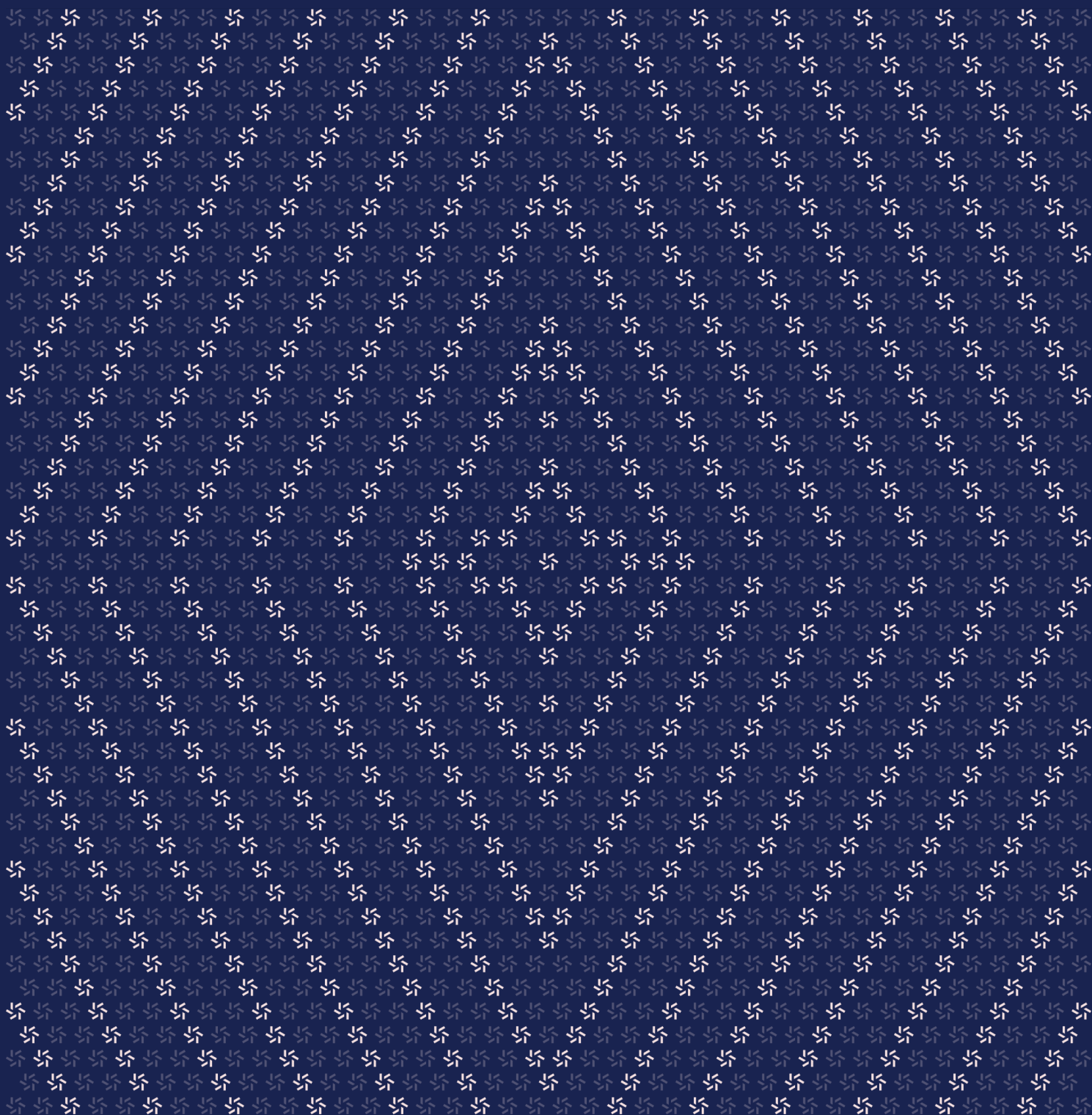


January 21, 2026

---

# Lighter (EVM)

## Smart Contract Security Assessment



## Contents

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>About Zellic</b>                                                              | <b>4</b>  |
| <hr/>                                                                            |           |
| <b>1. Overview</b>                                                               | <b>4</b>  |
| 1.1. Executive Summary                                                           | 5         |
| 1.2. Goals of the Assessment                                                     | 5         |
| 1.3. Non-goals and Limitations                                                   | 5         |
| 1.4. Results                                                                     | 6         |
| <hr/>                                                                            |           |
| <b>2. Introduction</b>                                                           | <b>6</b>  |
| 2.1. About Lighter (EVM)                                                         | 7         |
| 2.2. Methodology                                                                 | 7         |
| 2.3. Scope                                                                       | 9         |
| 2.4. File Checksums                                                              | 9         |
| 2.5. Project Overview                                                            | 10        |
| 2.6. Project Timeline                                                            | 11        |
| <hr/>                                                                            |           |
| <b>3. Detailed Findings</b>                                                      | <b>11</b> |
| 3.1. L1 createOrder rejects spot-market indexes                                  | 12        |
| 3.2. Insufficient test coverage                                                  | 14        |
| 3.3. Default asset configs rely on manual migration during upgrades              | 16        |
| 3.4. Priority requests not revalidated against current L1 state during execution | 19        |
| <hr/>                                                                            |           |
| <b>4. Discussion</b>                                                             | <b>20</b> |
| 4.1. Notes on issues already handled                                             | 21        |

---

|           |                                 |           |
|-----------|---------------------------------|-----------|
| <b>5.</b> | <b>Threat Model</b>             | <b>23</b> |
| 5.1.      | Module: AdditionalZkLighter.sol | 24        |
| 5.2.      | Module: ZkLighter.sol           | 62        |

---

|           |                           |            |
|-----------|---------------------------|------------|
| <b>6.</b> | <b>Assessment Results</b> | <b>106</b> |
| 6.1.      | Disclaimer                | 107        |

## About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website [zellic.io](https://zellic.io) and follow [@zellic\\_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at [hello@zellic.io](mailto:hello@zellic.io).



## 1. Overview

### 1.1. Executive Summary

Zellic conducted a security assessment for Lighter from December 25th, 2025 to January 7th, 2026. During this engagement, our team analyzed the Lighter (EVM) codebase to identify security vulnerabilities, design flaws, and other issues that could impact its security posture.

---

### 1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are spot-market index ranges correctly validated across all priority-request flows and state transitions?
  - Can spot-market functionality introduce vulnerabilities or bypass existing security controls in the priority-request queue?
  - Can emergency exit mechanisms (desert mode) be exploited or misused to lock funds permanently?
  - Are authorization boundaries between on-chain and off-chain validation clearly defined and secure?
  - Do asset-configuration migrations during upgrades properly initialize all default assets without creating locked states?
  - Does the upgrade mechanism properly validate parameters and maintain access-control integrity?
  - Are state migrations during upgrades handled safely without introducing inconsistencies or fund loss?
  - Are initialization and upgrade functions protected against reinitialization or replay attacks?
- 

### 1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody
- Off-chain system components (sequencer, prover, indexer, zero-knowledge circuits)

We assume these off-chain components are functioning as intended and focus exclusively on the on-chain smart contract security.

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

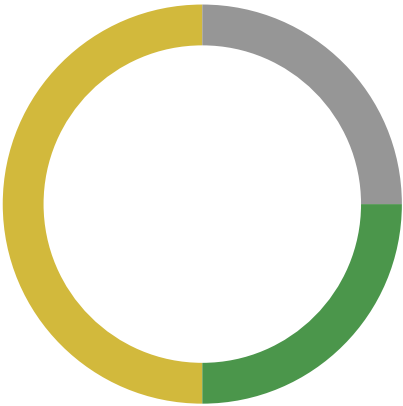
1.4. Results

During our assessment on the scoped Lighter (EVM) contracts, we discovered four findings. No critical issues were found. Two findings were of medium impact, one was of low impact, and the remaining finding was informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Lighter in the Discussion section (4.7).

Breakdown of Finding Impacts

| Impact Level                                                                                      | Count |
|---------------------------------------------------------------------------------------------------|-------|
|  Critical       | 0     |
|  High          | 0     |
|  Medium        | 2     |
|  Low           | 1     |
|  Informational | 1     |



## 2. Introduction

### 2.1. About Lighter (EVM)

Lighter contributed the following description of Lighter (EVM):

Lighter is a decentralized trading platform built for unmatched security and scale. It's the first exchange to offer verifiable order matching and liquidations while delivering best-in-class performance on par with traditional exchanges.

---

### 2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

**Basic coding mistakes.** Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

**Business logic errors.** Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

**Integration risks.** Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

**Code maturity.** We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. 7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

## 2.3. Scope

The engagement involved a review of the following targets:

### Lighter (EVM) Contracts

|            |                                                                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------------------------------|
| Type       | Solidity                                                                                                                           |
| Platform   | EVM-compatible                                                                                                                     |
| Target     | Only changes between 5cbd2d23..e6890b14                                                                                            |
| Repository | <a href="https://github.com/elliotech/lighter-contracts.git">https://github.com/elliotech/lighter-contracts.git</a>                |
| Version    | e6890b14a340321be161fb99d980c71c348541dc                                                                                           |
| Programs   | ZkLighter.sol<br>AdditionalZkLighter.sol<br>lib/Bytes.sol<br>lib/TxTypes.sol<br>ExtendableStorage.sol<br>Config.sol<br>Storage.sol |

## 2.4. File Checksums

This assessment covered Lighter (EVM) at commit hash e6890b14a340321be161fb99d980c71c348541dc. The SHA-256 checksums of the audited source files are listed below. File paths are relative to the contracts/ directory.

| File                    | SHA-256                                                          |
|-------------------------|------------------------------------------------------------------|
| AdditionalZkLighter.sol | ab3a4944ac609dd300c9f205bdc2d909974c227da1b57f9333ea1aca7524f936 |
| Config.sol              | d58837a79c3a51189707a24636408532881918ab5be3b82296384d7b60266a39 |
| ExtendableStorage.sol   | 462ad9a220c39a456e31ac7d5fa5bf7590e68195a6eadd4fd5eca061f94b91f8 |
| lib/Bytes.sol           | 8312145c6db2cc32c89e7b19685c91fb5d22c05a89fe05b3f32cf63a8b681788 |
| lib/TxTypes.sol         | 836b2d80238de6d7415efa2c8d3fff6c9a782d47ee0716d490627c5680d9429b |
| Storage.sol             | a4c6946df6be78289ade3f2fefc4dd034f5e26edad0a9f6a25337b32185a9845 |
| ZkLighter.sol           | afb22aba6f17d81b0c5707cf5a308ea3072edf5d4c136642605f86d0083dc9a8 |

The following script was used to compute these checksums, should the reader wish to reproduce them (run from the root of the repository at the audited commit):

```
#!/usr/bin/env zsh
files=(
  contracts/AdditionalZkLighter.sol
  contracts/Config.sol
  contracts/ExtendableStorage.sol
  contracts/lib/Bytes.sol
  contracts/lib/TxTypes.sol
  contracts/Storage.sol
  contracts/ZkLighter.sol
)
shasum -a 256 $files | awk '{printf "%-48s - %s\n", $2, $1}'
```

## 2.5. Project Overview

Zellic was contracted to perform a security assessment for a total of 2.2 person-weeks. The assessment was conducted by two consultants over the course of nine calendar days.

## Contact Information

---

The following project manager was associated with the engagement:

**Jacob Goreski**  
✈ Engagement Manager  
[jacob@zellic.io](mailto:jacob@zellic.io) ↗

---

The following consultants were engaged to conduct the assessment:

**Filipe Alves**  
✈ Engineer  
[filipe@zellic.io](mailto:filipe@zellic.io) ↗

**Quentin Lemauf**  
✈ Engineer  
[quentin@zellic.io](mailto:quentin@zellic.io) ↗

---

## 2.6. Project Timeline

The key dates of the engagement are detailed below.

**December 25, 2025**    Start of primary review period

---

**January 7, 2026**        End of primary review period

### 3. Detailed Findings

#### 3.1. L1 createOrder rejects spot-market indexes

|                   |                     |                 |        |
|-------------------|---------------------|-----------------|--------|
| <b>Target</b>     | AdditionalZkLighter |                 |        |
| <b>Category</b>   | Coding Mistakes     | <b>Severity</b> | Medium |
| <b>Likelihood</b> | Medium              | <b>Impact</b>   | Medium |

#### Description

The protocol uses different market index ranges for different market types:

- Perps markets use indexes  $\leq$  MAX\_PERPS\_MARKET\_INDEX (currently 254).
- Spot markets use indexes in the MIN\_SPOT\_MARKET\_INDEX..MAX\_SPOT\_MARKET\_INDEX range (currently 2048..4094).

However, AdditionalZkLighter::createOrder validates \_marketIndex only against MAX\_PERPS\_MARKET\_INDEX and reverts for any value greater than 254. As a result, every attempt to submit an L1 order for a spot market (which must use an index  $\geq$  2048) will revert and never be added to the priority queue.

```
function createOrder(
    uint48 _accountIndex,
    uint16 _marketIndex,
    uint48 _baseAmount,
    uint32 _price,
    uint8 _isAsk,
    uint8 _orderType
) external nonReentrant onlyActive {

    // ...

    if (_marketIndex > MAX_PERPS_MARKET_INDEX) {
        revert AdditionalZkLighter_InvalidMarketIndex();
    }

    // ...
}
```

#### Impact

Spot markets cannot accept L1 submitted orders via createOrder, effectively disabling the spot-order flow through the on-chain priority-request queue (“escape hatch”) that the

documentation describes as existing “to guarantee liveness and censorship resistance” by allowing users to submit critical operations on chain (with the Sequencer required to process them within a predefined time frame). See the documentation [here](#).

## Recommendations

Update createOrder market index validation to accept both perps and spot-market index ranges:

- `_marketIndex <= MAX_PERPS_MARKET_INDEX` or
- `MIN_SPOT_MARKET_INDEX <= _marketIndex <= MAX_SPOT_MARKET_INDEX`

## Remediation

This issue has been acknowledged by Lighter.

### 3.2. Insufficient test coverage

|                   |                |                 |        |
|-------------------|----------------|-----------------|--------|
| <b>Target</b>     | Project-wide   |                 |        |
| <b>Category</b>   | Protocol Risks | <b>Severity</b> | Medium |
| <b>Likelihood</b> | N/A            | <b>Impact</b>   | Medium |

#### Description

The test suite for the Lighter EVM contracts has notable gaps in coverage for critical functionality. While basic deposit, withdrawal, and batch-processing flows have some coverage, several important areas lack comprehensive testing.

The EIP-4844 blob-processing functions (`_pointEvaluationPrecompile` and `_processBlobs`) have limited direct test coverage: the happy path is exercised via `commitBatch`, but blob-verification edge cases and error conditions are untested. Similarly, `revertBatches` has no meaningful test coverage.

Desert-mode functionality, which handles emergency withdrawals, lacks comprehensive testing. Functions such as `performDesert` and `createExitCommitment` have no tests, and `cancelOutstandingDepositsForDesertMode` has only partial coverage with several important branches untested. The `updateStateRoot` function also has no direct tests, and upgrade-related state-root override logic (via `stateRootUpdates` during `commitBatch`) remains unverified.

Several delegation wrapper functions have limited or no direct testing: `registerAsset` and `updateAsset` are never called in tests (all validation logic remains untested); `depositBatch` has no direct coverage for batch-specific validations (array-length matching, empty arrays, `MAX_BATCH_DEPOSIT_LENGTH`); `createOrder` never tests market orders or `NIL_ORDER_BASE_AMOUNT`; and `burnShares` is missing tests for the `_accountIndex == _publicPoolIndex` equality check and unregistered caller `revert`. Proxy and upgrade mechanisms also have minimal coverage.

The deposit flow handles fee-on-transfer tokens by calculating `balanceAfter - balanceBefore` and crediting excess to the treasury, but this logic is never tested.

Additionally, the test suite lacks end-to-end (E2E) tests entirely. Given that critical paths in this protocol involve off-chain components (sequencer, ZK circuit, L2 state management), E2E tests are essential to verify the full flow from L1 priority-request creation through off-chain processing to final L1 execution — specifically the priority-request lifecycle, the batch-processing flow (`commitBatch` → `verifyBatch` → `executeBatches`), and the desert-mode activation and exit flow. Without E2E tests, interactions between L1 contracts and off-chain components remain unverified. See section [5.7](#) for further details.

## Impact

Insufficient test coverage increases the likelihood of bugs and vulnerabilities going unnoticed until after deployment. The untested areas include critical emergency functionality (desert mode), newer EVM features (EIP-4844 blobs), and state-management operations that are difficult to verify through manual review alone.

## Recommendations

We recommend building a comprehensive E2E test suite that covers the full life cycle of priority requests from L1 creation through off-chain processing to final execution, including the batch-processing flow and desert-mode activation. This would help verify the interactions between on-chain contracts and off-chain components that are critical to the protocol's security.

Additionally, unit test coverage should be expanded to include EIP-4844 blob-processing functions with edge cases and malformed blob data, batch reversion scenarios, desert-mode activation and fund-recovery flows, state-root upgrade flows with mock ZK proofs, all delegation wrapper functions (particularly `registerAsset`, `updateAsset`, `depositBatch` batch-specific logic, `createOrder` market orders, and `burnShares` edge cases), fee-on-transfer token handling, and negative tests for revert conditions in critical paths.

## Remediation

This issue has been acknowledged by Lighter.

### 3.3. Default asset configs rely on manual migration during upgrades

|                   |                |                 |     |
|-------------------|----------------|-----------------|-----|
| <b>Target</b>     | ZkLighter      |                 |     |
| <b>Category</b>   | Business Logic | <b>Severity</b> | Low |
| <b>Likelihood</b> | Low            | <b>Impact</b>   | Low |

#### Description

The function `_registerDefaultAssetConfigs()` is only called in `ZkLighter.initialize()` and not in `ZkLighter.upgrade()`. This helper registers the default assets and their parameters, including native ETH (`NATIVE_ASSET_INDEX = 1`) and USDC (`USDC_ASSET_INDEX = 3`).

When a proxy upgrade occurs via `Proxy.upgradeTarget()`, it calls `ZkLighter.upgrade()` rather than `initialize()`. If a live deployment is upgraded to the multiasset version without a separate migration step, the default asset configs would remain uninitialized. This is particularly fragile because ETH cannot be configured later using `ZkLighter.registerAssetConfig()`, as it explicitly rejects `NATIVE_ASSET_INDEX`:

```
// [...]
if (assetIndex == NATIVE_ASSET_INDEX || assetConfigs[assetIndex].tokenAddress
    != address(0)) {
    revert ZkLighter_InvalidAssetIndex();
}
// [...]
```

Additionally, the `_deposit()` function in `AdditionalZkLighter` does not validate that the native ETH asset configuration is properly initialized before performing arithmetic operations with `tickSize`. For non-native assets, uninitialized configs are caught by checking `tokenAddress == address(0)`. However, native ETH legitimately uses `tokenAddress = address(0)`, so this validation is skipped. The function then uses `tickSize` directly in modulo and division operations. If `tickSize == 0`, both operations would cause a division-by-zero panic.

#### Impact

If native ETH's asset configuration is not properly set, all native ETH deposits would panic with division by zero (not a clean revert), users would receive unhelpful error messages, and ETH deposit functionality would be completely broken. Unlike USDC, which fails safely with `InvalidAssetIndex`, native ETH has no fallback validation.

## Recommendations

We recommend keeping `_registerDefaultAssetConfigs()` permanently in the `upgrade()` function. This would ensure that future upgrades automatically initialize default asset configs without requiring manual intervention.

Lighter team has acknowledged this issue and provided the following response:

We already performed the upgrade by calling `_registerDefaultAssetConfigs()` in the upgrade function and then removed it from the function.

This addressed the immediate risk for the current upgrade. However, this differs from our recommendation in that the function call was temporary—it was removed after the upgrade completed.

For defense in depth, also consider adding explicit validation for native ETH configuration in `AdditionalZkLighter::_deposit()`:

```
function _deposit(address[] memory _to, uint16 _assetIndex,
    TxTypes.RouteType _routeType, uint256[] memory _amount) internal {
    AssetConfig memory assetConfig = assetConfigs[_assetIndex];
    if (_assetIndex != NATIVE_ASSET_INDEX) {
        if (assetConfig.tokenAddress == address(0)) {
            revert AdditionalZkLighter_InvalidAssetIndex();
        }
    }
    if (_assetIndex == NATIVE_ASSET_INDEX) {
        if (assetConfig.tickSize == 0) {
            revert AdditionalZkLighter_InvalidAssetIndex();
        }
    }
    } else {
        if (assetConfig.tokenAddress == address(0)) {
            revert AdditionalZkLighter_InvalidAssetIndex();
        }
    }
    if (msg.value != 0) {
        revert AdditionalZkLighter_InvalidDepositAmount();
    }
}
```

Additionally, the test suite has the specific test for the ZkLighter upgrade skipped (`it.skip('upgrade zkLighter', ...)` in `test/upgradeable/Proxy.test.ts`), and no migration test exists that simulates upgrading from the old version to the new multiasset version. We recommend implementing proper test coverage for upgrade scenarios to ensure default asset configs are correctly initialized during future upgrades.

## Remediation

This issue has been acknowledged by Lighter.

### 3.4. Priority requests not revalidated against current L1 state during execution

|                   |                                |                 |               |
|-------------------|--------------------------------|-----------------|---------------|
| <b>Target</b>     | ZkLighter, AdditionalZkLighter |                 |               |
| <b>Category</b>   | Business Logic                 | <b>Severity</b> | Informational |
| <b>Likelihood</b> | Low                            | <b>Impact</b>   | Informational |

#### Description

Priority requests are validated only at creation time on L1. Once a priority request is created and added to the queue, governance changes to asset configuration (such as `depositCapTicks` or `minDepositTicks`) do not affect its processing. The on-chain batch life-cycle functions (`commitBatch`, `verifyBatch`, `executeBatches`) do not revalidate the current L1 configuration state before executing priority operations.

For example, if a user creates a deposit request when `depositCapTicks` allows for large deposits, and governance then reduces `depositCapTicks`, the deposit would still complete successfully even though it exceeds the new cap. These snapshot semantics are likely intentional, as users should be able to rely on their valid requests being processed.

It is worth noting that `withdrawalsEnabled` cannot be disabled once enabled (the code enforces a one-way transition from 0 to 1), so the scenario of pausing withdrawals does not apply to in-flight requests.

#### Impact

This behavior is likely intentional, as users should be able to rely on their valid requests being processed regardless of later governance changes. However, governance should be aware that configuration changes only affect new priority requests, not in-flight ones. Users monitoring governance actions could also front-run configuration changes by creating requests before new restrictions take effect, though this does not result in fund loss.

#### Recommendations

We recommend documenting the snapshot validation behavior so that governance understands configuration changes only affect new priority requests. Additionally, it should be clarified whether the sequencer and ZK circuit also use snapshot semantics when processing priority requests, to ensure consistent behavior across L1 and off-chain components.

## Remediation

This issue has been acknowledged by Lighter.

## 4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

### 4.1. Notes on issues already handled

The following are several notes on issues that are already handled by the off-chain process or prior upgrade.

#### Parameter-locked upgrade ( ) via a hardcoded commitment hash

As shown below, `ZkLighter.upgrade()` gates upgrades by requiring `keccak256(upgradeParameters)` to match a hardcoded commitment:

```
bytes32 upgradeParametersHash = keccak256(upgradeParameters);
// Commits to 0 address for _additionalZkLighter, _desertVerifier and
// _stateRootUpgradeVerifier
bytes32 initializationParametersCommitment
    = 0x46700b4d40ac5c35af2c22dda2787a91eb567b06c924a8fb8ae9a05b20c08c21;
if (upgradeParametersHash != initializationParametersCommitment) {
    revert ZkLighter_InvalidUpgradeParameters();
}
```

At the time of writing, the commitment corresponds to the “all-zero addresses” parameter set (i.e., `abi.encode(address(0), address(0), address(0))`). This effectively disables address updates via the current `upgrade()` entry point (calls either `revert` or performs no state changes).

This is worth noting because it can look like a broken or inverted check when reviewing the code in isolation.

The Lighter team notes the following:

This is an intentional “commitment-based” upgrade flow. The team plans to handle upgrades by deploying a specific upgrade implementation where the hard-coded commitment is updated to the hash of the intended new addresses.

#### Reliance of `AdditionalZkLighter.changePubKey` on circuit-level authorization

As shown below, `AdditionalZkLighter.changePubKey` only enforces that `msg.sender` is a registered master account:

```
uint48 _masterAccountIndex = getAccountIndexFromAddress(msg.sender);  
if (_masterAccountIndex == NIL_ACCOUNT_INDEX) {  
    revert AdditionalZkLighter_AccountIsNotRegistered();  
}
```

However, there is no on-chain access-control check that ties the requested `_accountIndex` to `msg.sender`. As written, any registered master account can enqueue a `ChangePubKey` priority request for an arbitrary `_accountIndex`.

The Lighter team notes the following:

Authorization is checked in the circuits; account indices and master account indices are not tied together in the contracts.

### Requirement of `ZkLighter.performDesert` to provide the full per-asset balance (no partial desert exit)

As shown below, `ZkLighter.performDesert` requires callers to provide `_totalBaseAmount`, which is incorporated into the public commitment verified by `desertVerifier`:

```
bytes32 commitment = createExitCommitment(  
    uint256(stateRoot),  
    _accountIndex,  
    _l1Address,  
    _assetIndex,  
    _totalBaseAmount  
);
```

Given that `_totalBaseAmount` is user-supplied and there is no on-chain check that it matches the account's full balance, an incorrect value could permanently lock the remaining funds for that asset/account.

The Lighter team notes the following:

Users must withdraw their entire balance for that asset for that account index; partial desert exits are not supported.

### If an ERC-20 overtransfers, AdditionalZkLighter.\_deposit can leave unaccounted dust

When depositing ERC-20s via `AdditionalZkLighter._deposit`, the implementation compares `balanceAfter` vs `balanceBefore + totalAmount` to detect nonstandard tokens. In particular, if the token transfers *more* than `totalAmount`, the excess is credited to the treasury:

```
} else if (balanceAfter > balanceBefore + totalAmount) {  
    uint256 excessAmount = balanceAfter - (balanceBefore + totalAmount);  
    uint64 baseExcessAmount = SafeCast.toUint64(excessAmount  
        / assetConfig.tickSize);  
    increaseBalanceToWithdraw(TREASURY_ACCOUNT_INDEX, _assetIndex,  
        baseExcessAmount);  
}
```

However, because the credited amount is rounded down by `assetConfig.tickSize`, any remainder (`excessAmount % assetConfig.tickSize`) is left in the contract and is not represented in the tick-based accounting. This means that up to `assetConfig.tickSize - 1` (in token units) can be left as dust per interaction if this branch is ever triggered, and it can accumulate over time.

The Lighter team notes the following:

Intended and acknowledged – expected not to trigger for supported assets.

## 5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

### 5.1. Module: AdditionalZkLighter.sol

**Function:** `updateStateRoot(StoredBatchInfo calldata _lastStoredBatch, bytes32 _stateRoot, bytes32 _validiumRoot, bytes calldata proof)`

This function allows active validators to upgrade the state root after all batches have been executed. It verifies a zero-knowledge proof before updating critical state variables.

#### Inputs

- `_lastStoredBatch`
  - **Control:** Validator control.
  - **Constraints:** Must match the stored batch hash at `storedBatchHashes[committedBatchesCount]`.
  - **Impact:** Used to verify the current state before upgrade.
- `_stateRoot`
  - **Control:** Validator control.
  - **Constraints:** Must be verified by the state-root upgrade verifier proof.
  - **Impact:** The new state root that will replace the current one.
- `_validiumRoot`
  - **Control:** Validator control.
  - **Constraints:** Must be verified by the state-root upgrade verifier proof.
  - **Impact:** The new validium root that will replace the current one.
- `proof`
  - **Control:** Validator control.
  - **Constraints:** Must be a valid ZK proof that verifies the state transition from old to new roots.
  - **Impact:** The cryptographic proof ensuring the state transition is valid.

#### Branches and code coverage

##### Intended branches

- State root and validium root should be updated successfully when a valid proof is provided.
  - ☐ Test coverage (not explicitly tested — edge-case feature)
- The state-root upgrade verifier should be set to address (0) after successful verification.
  - ☐ Test coverage (not explicitly tested)

### Negative behavior

- Revert when the caller is not an active validator.
  - ☐ Negative test: not explicitly tested
- Revert when stored batch information does not match.
  - ☐ Negative test: not explicitly tested
- Revert when there are pending verified requests to execute.
  - ☐ Negative test: not explicitly tested
- Revert when the state-root upgrade verifier is not set.
  - ☐ Negative test: not explicitly tested
- Revert when ZK proof verification fails.
  - ☐ Negative test: not explicitly tested

### Function call analysis

- `governance.isActiveValidator(msg.sender)`
  - **What is controllable?** `msg.sender` by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if sender is not an active validator.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Reverts the entire transaction, preventing unauthorized state-root updates.
- `hashStoredBatchInfo(_lastStoredBatch)`
  - **What is controllable?** The entire `_lastStoredBatch` struct by validator.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
The return value is compared against stored hash. Mismatch causes a revert.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Pure function, no unusual control flow.
- `stateRootUpgradeVerifier.Verify(proof, inputs)`
  - **What is controllable?** `proof` is fully controlled by the validator, and `inputs` is derived from state roots.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Returns boolean. If false, the transaction reverts. A validator cannot force a

true result without valid proof.

- **What happens if it reverts, reenters or does other unusual control flow?** If it reverts, the entire transaction fails. It is an external call to the verifier contract but protected by nonReentrant.

**Function: deposit(address \_to, uint16 \_assetIndex, TxTypes.RouteType \_routeType, uint256 \_amount)**

This function allows users to deposit ETH or ERC-20 assets to ZkLighter. It validates deposit amounts, handles both native and ERC-20 tokens, and registers the deposit in the priority queue.

## Inputs

- `_to`
  - **Control:** User control.
  - **Constraints:** Cannot be `address(0)` — validated in `_deposit`.
  - **Impact:** Recipient address for deposit on L2.
- `_assetIndex`
  - **Control:** User control.
  - **Constraints:** Must be valid (`NATIVE_ASSET_INDEX` or configured ERC-20). See `validateAssetIndex`.
  - **Impact:** Determines which asset to deposit.
- `_routeType`
  - **Control:** User control.
  - **Constraints:** Type-safe enum (`RouteType`).
  - **Impact:** Routes deposit to perps or spot.
- `_amount`
  - **Control:** User control.
  - **Constraints:** Must be `>= minDeposit`, aligned with `tickSize`, and not exceed `depositCap`.
  - **Impact:** Amount to deposit.

## Branches and code coverage

### Intended branches

- Native asset (ETH) deposits should succeed when `msg.value` matches `_amount`.
- ☑ Test coverage: `ZkLighter.t.sol::L-156`  
(`test_deposit_eth_fail_and_success`)

- ERC-20 deposits should work with exact amounts.
  - ☑ Test coverage: `ZkLighter.t.sol::L-101` (`test_deposit_usdc_fail_and_success`)
- Account should be auto-registered if depositing to unregistered address.
  - ☑ Test coverage: `ZkLighter.test.ts::L-701` (via `_deposit`)
- Deposit should be added to priority queue.
  - ☑ Test coverage: `ZkLighter.test.ts::L-711` (`NewPriorityRequest` event, via `_deposit`)

### Negative behavior

- Revert when non-native asset has no configured `tokenAddress`.
  - ☑ Negative test: `ZkLighter.t.sol::L-110`
- Revert when depositing ERC-20 with nonzero `msg.value`.
  - ☑ Negative test: `ZkLighter.t.sol::L-126`
- Revert when amount is `< minDeposit` or not aligned with `tickSize`.
  - ☑ Negative test: `ZkLighter.t.sol::L-122`
- Revert when recipient is `address(0)`.
  - ☑ Negative test: `ZkLighter.test.ts::L-686`
- Revert when `ETH msg.value != _amount`.
  - ☑ Negative test: `ZkLighter.t.sol::L-172`
- Revert when ERC-20 transfer results in less balance than expected.
  - ☐ Negative test: not explicitly tested (fee-on-transfer tokens)
- Revert when `balanceAfter` exceeds `depositCap`.
  - ☑ Negative test: `ZkLighter.test.ts::L-692`

### Function call analysis

This function wraps `_deposit` with a single recipient. All deposit logic (validation, transfers, priority queue) is handled in `_deposit`. See `_deposit` for detailed function call analysis.

### Function: `depositBatch(uint64[] calldata _amount, address[] calldata _to, uint48[] calldata _accountIndex)`

This function allows batch deposits of USDC to ZkLighter for multiple users in a single transaction. It is hardcoded to use `USDC_ASSET_INDEX` and `RouteType.Perps`.

## Inputs

- `_amount`
  - **Control:** User control.
  - **Constraints:** Array length must match `_to` and `_accountIndex`, be  $> 0$  and  $\leq \text{MAX\_BATCH\_DEPOSIT\_LENGTH}$ . Each amount validated in `_deposit`.
  - **Impact:** USDC amounts to deposit for each recipient.
- `_to`
  - **Control:** User control.
  - **Constraints:** Array length must match `_amount` and `_accountIndex`. Each address cannot be `address(0)` — validated in `_deposit`.
  - **Impact:** Recipient addresses for deposits.
- `_accountIndex`
  - **Control:** User control.
  - **Constraints:** Array length must match `_amount` and `_to`. Currently unused but validated for length.
  - **Impact:** Reserved for future use.

## Branches and code coverage

### Intended branches

- USDC batch deposits should succeed with valid arrays.
  - ☐ Test coverage: not tested (core logic tested via `_deposit`, but batch iteration with multiple recipients not tested)
- Account should be auto-registered if depositing to unregistered address.
  - ☐ Test coverage: not tested (single registration tested via `_deposit`, but multiple registrations in batch not tested)
- Deposit should be added to priority queue for each recipient.
  - ☐ Test coverage: not tested (single priority request tested via `_deposit`, but multiple in batch not tested)

### Negative behavior

- Revert when array lengths do not match.
  - ☐ Negative test: not tested (batch-specific)
- Revert when arrays are empty.
  - ☐ Negative test: not tested (batch-specific)
- Revert when array length exceeds `MAX_BATCH_DEPOSIT_LENGTH`.

- ☐ Negative test: not tested (batch-specific)
  - Revert when amount is < minDeposit or not aligned with tickSize.
- ☒ Negative test: indirectly via \_deposit — ZkLighter.t.sol::L-122
  - Revert when recipient is address(0).
- ☒ Negative test: indirectly via \_deposit — ZkLighter.test.ts::L-686
  - Revert when ERC-20 transfer fails.
- ☒ Negative test: indirectly via \_deposit — ZkLighter.t.sol::L-115

## Function call analysis

This function wraps \_deposit with multiple recipients, hardcoded to USDC and perps. All deposit logic (validation, transfers, priority queue) is handled in \_deposit. See \_deposit for detailed function call analysis.

### Function: changePubKey(uint48 \_accountIndex, uint8 \_apiKeyIndex, bytes calldata \_pubKey)

Allows registered users to change the ZkLighter public key for an account API key slot. Validates public key format and Goldilocks field constraints. Protected by nonReentrant and onlyActive modifiers.

## Inputs

- \_accountIndex
  - **Control:** User control (caller must be registered master account).
  - **Constraints:** Must be <= MAX\_ACCOUNT\_INDEX.
  - **Impact:** Account index whose public key will be changed.
- \_apiKeyIndex
  - **Control:** User control.
  - **Constraints:** Must be <= MAX\_API\_KEY\_INDEX.
  - **Impact:** API key slot to update (multiple keys per account).
- \_pubKey
  - **Control:** User control.
  - **Constraints:** Must be exactly PUB\_KEY\_BYTES\_SIZE (40 bytes), cannot be all zeros, each 8-byte element must be < GOLDILOCKS\_MODULUS.
  - **Impact:** New public key for signing ZkLighter operations.

## Branches and code coverage

### Intended branches

- Public key change should succeed with valid parameters.
  - ☑ Test coverage: `ZkLighter.test.ts::L-837`
- Priority request should be added to queue.
  - ☑ Test coverage: `ZkLighter.test.ts::L-173` (helper verifies `NewPriorityRequest` event)
- Caller must be registered master account.
  - ☑ Test coverage: `ZkLighter.test.ts::L-836` (implicit — deposit registers account first)

### Negative behavior

- Revert when account index is invalid ( $> \text{MAX\_ACCOUNT\_INDEX}$ ).
  - ☑ Negative test: `ZkLighter.test.ts::L-808`
- Revert when API key index is invalid ( $> \text{MAX\_API\_KEY\_INDEX}$ ).
  - ☑ Negative test: `ZkLighter.test.ts::L-811`
- Revert when public key length is incorrect.
  - ☑ Negative test: `ZkLighter.test.ts::L-814`
- Revert when public key is all zeros.
  - ☑ Negative test: `ZkLighter.test.ts::L-817`
- Revert when any public key element is  $\geq \text{GOLDILOCKS\_MODULUS}$ .
  - ☑ Negative test: `ZkLighter.test.ts::L-823`
- Revert when caller is not registered.
  - ☑ Negative test: `ZkLighter.test.ts::L-831`

## Function call analysis

- `getAccountIndexFromAddress(msg.sender)`
  - **What is controllable?** `msg.sender` is the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns master-account index from stored mapping. If `NIL_ACCOUNT_INDEX`, reverts. Cannot be manipulated — mapping is set during registration.
  - **What happens if it reverts, reenters or does other unusual control flow?** View function, no unusual control flow.
- `TxTypes.writeChangePubKeyPubDataForPriorityQueue(_tx)`

- **What is controllable?** All `_tx` struct parameters by caller.
- **If the return value is controllable, how is it used and how can it go wrong?**  
Returns encoded bytes for priority queue. Malformed encoding could cause L2 processing issues.
- **What happens if it reverts, reenters or does other unusual control flow?**  
Library function, no revert conditions.
- `addPriorityRequest`
  - **What is controllable?** `pubData` derived from user inputs.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — adds to priority queue. Queue corruption could prevent L2 from processing key change.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Internal function, no revert conditions.

**Function:** `registerAsset(uint8 _l1Decimals, uint8 _decimals, bytes32 _symbol, TxTypes.RegisterAsset calldata _params)`

Governor-only function to register new assets in ZkLighter. Validates asset parameters and adds a priority request for L2 registration. Protected by `nonReentrant` and `onlyActive` modifiers.

## Inputs

- `_l1Decimals`
  - **Control:** Governor control.
  - **Constraints:** Metadata only — not validated, passed to event/L2.
  - **Impact:** L1 decimal count for reference.
- `_decimals`
  - **Control:** Governor control.
  - **Constraints:** Metadata only — not validated, passed to event/L2.
  - **Impact:** Lighter system decimal count.
- `_symbol`
  - **Control:** Governor control.
  - **Constraints:** Metadata only — not validated, passed to event/L2.
  - **Impact:** Asset symbol for identification.
- `_params (TxTypes.RegisterAsset)`
  - **Control:** Governor control.
  - **Constraints:** Asset must have `tokenAddress` configured (except `NATIVE_ASSET_INDEX`). Extension multiplier must match existing config. L2 amounts must be `> 0` and `<= MAX_DEPOSIT_CAP_TICKS`. Margin mode:

Enabled for USDC only, Disabled for others.

- **Impact:** Contains `assetIndex`, `extensionMultiplier`, `minL2TransferAmount`, `minL2WithdrawalAmount`, `marginMode`.

## Branches and code coverage

### Intended branches

- Asset should be registered successfully with valid parameters.
  - ☐ Test coverage: not tested
- Extension multiplier must match preconfigured value in `assetConfigs`.
  - ☐ Test coverage: not tested
- USDC must have margin mode `Enabled`; other assets must have `Disabled`.
  - ☐ Test coverage: not tested
- Priority request should be added to queue.
  - ☐ Test coverage: not tested

### Negative behavior

- Revert when caller is not governor.
  - ☐ Negative test: not tested (governor check tested elsewhere)
- Revert when asset index is invalid for non-native assets.
  - ☐ Negative test: not tested
- Revert when extension multiplier does not match config.
  - ☐ Negative test: not tested
- Revert when `minL2TransferAmount` is 0 or > `MAX_DEPOSIT_CAP_TICKS`.
  - ☐ Negative test: not tested
- Revert when `minL2WithdrawalAmount` is 0 or > `MAX_DEPOSIT_CAP_TICKS`.
  - ☐ Negative test: not tested
- Revert when margin mode is invalid value.
  - ☐ Negative test: not tested
- Revert when margin mode is `Enabled` for non-USDC asset.
  - ☐ Negative test: not tested
- Revert when margin mode is `Disabled` for USDC.
  - ☐ Negative test: not tested

## Function call analysis

- `governance.requireGovernor(msg.sender)`
  - **What is controllable?** `msg.sender` is the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if not governor. If bypassed, unauthorized caller could register malicious assets.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Reverts entire transaction.
- `TxTypes.writeRegisterAssetPubDataForPriorityQueue(_params)`
  - **What is controllable?** All parameters by governor.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Returns encoded bytes for priority queue. Malformed encoding could cause L2 processing issues.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Library function, no revert conditions.
- `TxTypes.writeRegisterAssetPubDataForPriorityQueueWithMetadata(...)`
  - **What is controllable?** All parameters by governor.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Returns encoded bytes with metadata for event emission. Incorrect encoding could mislead off-chain indexers.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Library function, no revert conditions.
- `addPriorityRequest`
  - **What is controllable?** All data derived from governor inputs.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — adds to priority queue. Queue corruption could prevent L2 from processing asset registration.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Internal function, no revert conditions.

## Function: `updateAsset(TxTypes.UpdateAsset calldata _params)`

Governor-only function to update parameters of existing assets. Validates asset exists and checks constraints similar to `registerAsset`. Protected by `nonReentrant` and `onlyActive` modifiers.

## Inputs

- `_params(TxTypes.UpdateAsset)`
  - **Control:** Governor control.

- **Constraints:** Asset index must be valid (see `validateAssetIndex`). L2 amounts must be  $> 0$  and  $\leq \text{MAX\_DEPOSIT\_CAP\_TICKS}$ . Margin mode: Enabled for USDC only, Disabled for others.
- **Impact:** Contains `assetIndex`, `minL2TransferAmount`, `minL2WithdrawalAmount`, `marginMode`.

## Branches and code coverage

### Intended branches

- Asset parameters should be updated successfully with valid parameters.
  - ☐ Test coverage: not tested
- Asset index must be validated before updating.
  - ☐ Test coverage: not tested (validated via `validateAssetIndex`)
- USDC must have margin mode Enabled; other assets must have Disabled.
  - ☐ Test coverage: not tested
- Priority request should be added to queue.
  - ☐ Test coverage: not tested

### Negative behavior

- Revert when caller is not governor.
  - ☐ Negative test: not tested (governor check tested elsewhere)
- Revert when asset index is invalid.
  - ☐ Negative test: not tested
- Revert when `minL2TransferAmount` is 0 or  $> \text{MAX\_DEPOSIT\_CAP\_TICKS}$ .
  - ☐ Negative test: not tested
- Revert when `minL2WithdrawalAmount` is 0 or  $> \text{MAX\_DEPOSIT\_CAP\_TICKS}$ .
  - ☐ Negative test: not tested
- Revert when margin mode is invalid value.
  - ☐ Negative test: not tested
- Revert when margin mode is Enabled for non-USDC asset.
  - ☐ Negative test: not tested
- Revert when margin mode is Disabled for USDC.
  - ☐ Negative test: not tested

## Function call analysis

- `governance.requireGovernor(msg.sender)`
  - **What is controllable?** `msg.sender` is the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if not governor. If bypassed, unauthorized caller could modify asset parameters.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Reverts entire transaction.
- `validateAssetIndex`
  - **What is controllable?** Asset index by governor.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if invalid. If bypassed, could update non-existent assets.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Reverts if asset not configured.
- `TxTypes.writeUpdateAssetPubDataForPriorityQueue(_params)`
  - **What is controllable?** All parameters by governor.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Returns encoded bytes for priority queue. Malformed encoding could cause L2 processing issues.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Library function, no revert conditions.
- `addPriorityRequest`
  - **What is controllable?** All data derived from governor inputs.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — adds to priority queue. Queue corruption could prevent L2 from processing asset update.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Internal function, no revert conditions.

**Function:** `createMarket(uint8 _size_decimals, uint8 _price_decimals, bytes32 _symbol, TxTypes.CreateMarket calldata _params)`

This function is only available to the governor and allows creation of new markets (perps or spot). It performs extensive validation of market parameters including fees, margin fractions, interest rates, and order limits.

## Inputs

- `_size_decimals`

- **Control:** Governor control.
  - **Constraints:** Metadata only, no direct validation.
  - **Impact:** Number of decimals for order size display.
- `_price_decimals`
  - **Control:** Governor control.
  - **Constraints:** Metadata only, no direct validation.
  - **Impact:** Number of decimals for price display.
- `_symbol`
  - **Control:** Governor control.
  - **Constraints:** In bytes32 format, metadata only.
  - **Impact:** Market symbol for identification.
- `_params`
  - **Control:** Governor control.
  - **Constraints:** Complex validation via `!validateCreateMarketParams`.
  - **Impact:** Defines the market's operational configuration including fees, margin requirements, interest rates, order limits, and other type-specific parameters that will be enforced on-chain.

## Branches and code coverage

### Intended branches

- Perps market should be created with valid parameters.
  - ☒ Test coverage: `test ZkLighter.test.ts::L-943`
- Spot market should be created with valid parameters and different base/quote assets.
  - ☒ Test coverage: `test ZkLighter.test.ts::L-1120`

### Negative behavior

- Revert when the caller is not the governor.
  - ☐ Negative test (not explicitly tested)

## Function call analysis

- `governance.requireGovernor(msg.sender)`
  - **What is controllable?** `msg.sender` by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if not governor.

- **What happens if it reverts, reenters or does other unusual control flow?**  
Prevents unauthorized market creation.
- `validateCreateMarketParams(_params)`
  - **What is controllable?** All parameters by the governor.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Internal view function with comprehensive validation. Reverts on any invalid parameter. See `validateCreateMarketParams` for detailed analysis.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
View function, no reentrancy.

### Function: `updateMarket(TxTypes.UpdateMarket calldata _params)`

This function is only available to the governor and allows updating parameters of existing markets. It validates market status and parameters similar to `createMarket`.

### Inputs

- `_params`
  - **Control:** Governor control.
  - **Constraints:** Complex validation via `validateUpdateMarketParams`.
  - **Impact:** Contains `marketIndex`, `marketType`, and `marketData` with updated parameters.

### Branches and code coverage

#### Intended branches

- Perps market parameters should be updated successfully.
  - ☒ Test coverage: `test ZkLighter.test.ts::L-1270`
- Spot market parameters should be updated successfully.
  - ☒ Test coverage: `test ZkLighter.test.ts::L-1407`
- A priority request should be added to the queue.
  - ☒ Test coverage: Verified in tests `ZkLighter.test.ts::L-1270` and `ZkLighter.test.ts::L-1407`

#### Negative behavior

- Revert when the caller is not the governor.
  - ☐ Negative test (not explicitly tested)

## Function call analysis

- `governance.requireGovernor(msg.sender)`
  - **What is controllable?** `msg.sender` is determined by the caller, but the function reverts if not governor.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value — reverts if not governor.
  - **What happens if it reverts, reenters or does other unusual control flow?** Prevents unauthorized updates.
- `validateUpdateMarketParams(_params)`
  - **What is controllable?** All parameters by the governor.
  - **If the return value is controllable, how is it used and how can it go wrong?** Internal pure function validates all parameters. Reverts on invalid data. See `validateUpdateMarketParams` for detailed analysis.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure function, no reentrancy.
- `TxTypes.writeUpdateMarketPubDataForPriorityQueue(_params)`
  - **What is controllable?** All parameters by the governor.
  - **If the return value is controllable, how is it used and how can it go wrong?** Encodes market update data into bytes for the priority queue. Pure function, deterministic encoding.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure function, no reentrancy.
- `addPriorityRequest(TxTypes.PriorityPubDataTypeL1UpdateMarket, pubdata, pubdata)`
  - **What is controllable?** All encoded data by the governor.
  - **If the return value is controllable, how is it used and how can it go wrong?** Adds the market update to the priority request queue for L1 processing. See `addPriorityRequest` for detailed analysis.
  - **What happens if it reverts, reenters or does other unusual control flow?** No revert conditions.

## Function: `cancelAllOrders(uint48 _accountIndex)`

Allows registered users to cancel all orders for a specific account. Adds a priority request to L2 to cancel all pending orders. Protected by `nonReentrant` and `onlyActive` modifiers.

## Inputs

- `_accountIndex`

- **Control:** User control (caller must be registered master account).
- **Constraints:** Must be  $\leq$  MAX\_ACCOUNT\_INDEX.
- **Impact:** Account whose orders will be canceled.

## Branches and code coverage

### Intended branches

- Cancel request should succeed with valid parameters.
  - ☒ Test coverage: ZkLighter.test.ts::L-858
- Priority request should be added to queue.
  - ☒ Test coverage: ZkLighter.test.ts::L-141 (helper verifies NewPriorityRequest event)
- Caller must be registered master account.
  - ☒ Test coverage: ZkLighter.test.ts::L-857 (implicit — deposit registers account first)

### Negative behavior

- Revert when account index is invalid ( $>$  MAX\_ACCOUNT\_INDEX).
  - ☒ Negative test: ZkLighter.test.ts::L-850
- Revert when caller is not registered.
  - ☒ Negative test: ZkLighter.test.ts::L-843

## Function call analysis

- `getAccountIndexFromAddress(msg.sender)`
  - **What is controllable?** `msg.sender` is the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns master-account index from stored mapping. If `NIL_ACCOUNT_INDEX`, reverts. Cannot be manipulated — mapping is set during registration.
  - **What happens if it reverts, reenters or does other unusual control flow?** View function, no unusual control flow.
- `TxTypes.writeCancelAllOrdersPubDataForPriorityQueue(_tx)`
  - **What is controllable?** `accountIndex` and `masterAccountIndex` by caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns encoded bytes for priority queue. Malformed encoding could cause L2 processing issues.
  - **What happens if it reverts, reenters or does other unusual control flow?** Library function, no revert conditions.

- `addPriorityRequest`
  - **What is controllable?** `pubData` derived from user inputs.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value — adds to priority queue. Queue corruption could prevent L2 from processing cancellation.
  - **What happens if it reverts, reenters or does other unusual control flow?** Internal function, no revert conditions.

**Function:** `withdraw(uint48 _accountIndex, uint16 _assetIndex, Tx-Types.RouteType _routeType, uint64 _baseAmount)`

Allows registered users to withdraw ETH or ERC-20 assets from ZkLighter. Validates withdrawal parameters and adds a priority request for L2 processing. Protected by `nonReentrant` and `onlyActive` modifiers.

## Inputs

- `_accountIndex`
  - **Control:** User control (caller must be registered master account).
  - **Constraints:** None on-chain — should be validated by ZK circuit/sequencer to ensure caller owns the account.
  - **Impact:** Account to withdraw from.
- `_assetIndex`
  - **Control:** User control.
  - **Constraints:** Must be valid asset index (see `validateAssetIndex`). Withdrawals must be enabled for asset.
  - **Impact:** Asset to withdraw.
- `_routeType`
  - **Control:** User control.
  - **Constraints:** Must be Perps or Spot.
  - **Impact:** Route to withdraw from.
- `_baseAmount`
  - **Control:** User control.
  - **Constraints:** Must be  $> 0$  and  $\leq \text{depositCapTicks}$ .
  - **Impact:** Amount to withdraw in ticks.

## Branches and code coverage

### Intended branches

- ETH withdrawals should work correctly.
  - ☑ Test coverage: `ZkLighter.test.ts::L-790`
- ERC-20 withdrawals should work correctly.
  - ☑ Test coverage: `ZkLighter.test.ts::L-762`
- Priority request should be added to queue.
  - ☑ Test coverage: `ZkLighter.test.ts::L-94` (helper verifies `NewPriorityRequest` event)
- Caller must be registered master account.
  - ☑ Test coverage: `ZkLighter.test.ts::L-761` (implicit — deposit registers account first)

### Negative behavior

- Revert when asset index is invalid.
  - ☑ Negative test: `ZkLighter.test.ts::L-798`
- Revert when withdrawals are not enabled for asset.
  - ☑ Negative test: `ZkLighter.t.sol::L-261` (LIT withdrawals disabled initially)
- Revert when amount is 0 or > `depositCapTicks`.
  - ☑ Negative test: `ZkLighter.test.ts::L-743` (amount = 0), `ZkLighter.test.ts::L-749` (amount too large)
- Revert when route type is not Perps or Spot.
  - ☑ Negative test: `ZkLighter.test.ts::L-752`
- Revert when caller is not registered.
  - ☑ Negative test: `ZkLighter.test.ts::L-754`

### Function call analysis

- `getAccountIndexFromAddress(msg.sender)`
  - **What is controllable?** `msg.sender` is the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns master-account index from stored mapping. If `NIL_ACCOUNT_INDEX`, reverts. Cannot be manipulated — mapping is set during registration.
  - **What happens if it reverts, reenters or does other unusual control flow?** View function, no unusual control flow.
- `TxTypes.writeWithdrawPubDataForPriorityQueue(_tx)`
  - **What is controllable?** All `_tx` struct parameters by caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns encoded bytes for priority queue. Malformed encoding could cause

L2 withdrawal processing issues.

- **What happens if it reverts, reenters or does other unusual control flow?**

Library function, no revert conditions.

- `addPriorityRequest`

- **What is controllable?** `pubData` derived from user inputs.

- **If the return value is controllable, how is it used and how can it go wrong?**

No return value — adds to priority queue. Queue corruption could prevent L2 from processing withdrawal.

- **What happens if it reverts, reenters or does other unusual control flow?**

Internal function, no revert conditions.

### Function: `createOrder(uint48 _accountIndex, uint16 _marketIndex, uint48 _baseAmount, uint32 _price, uint8 _isAsk, uint8 _orderType)`

Allows registered users to create orders (limit or market) on ZkLighter perps markets. Validates all order parameters before adding to priority queue. Protected by `nonReentrant` and `onlyActive` modifiers.

### Inputs

- `_accountIndex`

- **Control:** User control (caller must be registered master account).

- **Constraints:** Must be  $\leq$  `MAX_ACCOUNT_INDEX`.

- **Impact:** Account placing the order.

- `_marketIndex`

- **Control:** User control.

- **Constraints:** Must be  $\leq$  `MAX_PERPS_MARKET_INDEX`.

- **Impact:** Perps market to trade on.

- `_baseAmount`

- **Control:** User control.

- **Constraints:** Must be `NIL_ORDER_BASE_AMOUNT` or in range `[MIN_ORDER_BASE_AMOUNT, MAX_ORDER_BASE_AMOUNT]`.

- **Impact:** Order size in base units.

- `_price`

- **Control:** User control.

- **Constraints:** Must be in range `[MIN_ORDER_PRICE, MAX_ORDER_PRICE]`.

- **Impact:** Order price.

- `_isAsk`

- **Control:** User control.
  - **Constraints:** Must be 0 or 1.
  - **Impact:** Order side (0 = bid, 1 = ask).
- `_orderType`
  - **Control:** User control.
  - **Constraints:** Must be `LimitOrder` (1) or `MarketOrder` (2).
  - **Impact:** Type of order to create.

## Branches and code coverage

### Intended branches

- Order should be created successfully with valid parameters.
  - ☒ Test coverage: `ZkLighter.test.ts::L-899`
- Market orders should be created successfully.
  - ☐ Test coverage: not tested (only limit orders tested)
- `NIL_ORDER_BASE_AMOUNT` should be accepted as special value.
  - ☐ Test coverage: not tested
- Priority request should be added to queue.
  - ☒ Test coverage: `ZkLighter.test.ts::L-207` (helper verifies `NewPriorityRequest` event)
- Caller must be registered master account.
  - ☒ Test coverage: `ZkLighter.test.ts::L-898` (implicit — deposit registers account first)

### Negative behavior

- Revert when account index is invalid (`> MAX_ACCOUNT_INDEX`).
  - ☒ Negative test: `ZkLighter.test.ts::L-867`
- Revert when market index is invalid (`> MAX_PERPS_MARKET_INDEX`).
  - ☒ Negative test: `ZkLighter.test.ts::L-891`
- Revert when base amount is invalid (not `NIL` and outside valid range).
  - ☐ Negative test: not tested
- Revert when price is outside valid range.
  - ☒ Negative test: `ZkLighter.test.ts::L-879` (price = 0 rejected)
- Revert when `isAsk` is not 0 or 1.
  - ☒ Negative test: `ZkLighter.test.ts::L-873`

- Revert when order type is not LimitOrder or MarketOrder.
- ☒ Negative test: ZkLighter.test.ts:L-885
- Revert when caller is not registered.
- ☐ Negative test: not tested

## Function call analysis

- `getAccountIndexFromAddress(msg.sender)`
  - **What is controllable?** `msg.sender` is the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns master-account index from stored mapping. If `NIL_ACCOUNT_INDEX`, reverts. Cannot be manipulated — mapping is set during registration.
  - **What happens if it reverts, reenters or does other unusual control flow?** View function, no unusual control flow.
- `TxTypes.writeCreateOrderPubDataForPriorityQueue(_tx)`
  - **What is controllable?** All `_tx` struct parameters by caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns encoded bytes for priority queue. Malformed encoding could cause L2 order processing issues.
  - **What happens if it reverts, reenters or does other unusual control flow?** Library function, no revert conditions.
- `addPriorityRequest`
  - **What is controllable?** `pubData` derived from user inputs.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value — adds to priority queue. Queue corruption could prevent L2 from processing order.
  - **What happens if it reverts, reenters or does other unusual control flow?** Internal function, no revert conditions.

## Function: `burnShares(uint48 _accountIndex, uint48 _publicPoolIndex, uint64 _shareAmount)`

Allows registered users to burn shares of a public pool. Validates pool and share parameters before adding to priority queue. Protected by `nonReentrant` and `onlyActive` modifiers.

## Inputs

- `_accountIndex`
  - **Control:** User control (caller must be registered master account).

- **Constraints:** Must be  $\leq$  MAX\_ACCOUNT\_INDEX. Cannot equal \_publicPoolIndex.
  - **Impact:** Account burning shares.
- \_publicPoolIndex
  - **Control:** User control.
  - **Constraints:** Must be in range (MAX\_MASTER\_ACCOUNT\_INDEX, MAX\_ACCOUNT\_INDEX]. Cannot equal \_accountIndex.
  - **Impact:** Public pool to burn shares from.
- \_shareAmount
  - **Control:** User control.
  - **Constraints:** Must be in range [MIN\_POOL\_SHARES\_TO\_MINT\_OR\_BURN, MAX\_POOL\_SHARES\_TO\_MINT\_OR\_BURN].
  - **Impact:** Amount of shares to burn.

## Branches and code coverage

### Intended branches

- Shares should be burned successfully with valid parameters.
  - ☒ Test coverage: ZkLighter.test.ts::L-939
- Priority request should be added to queue.
  - ☒ Test coverage: ZkLighter.test.ts::L-236 (helper verifies NewPriorityRequest event)
- Caller must be registered master account.
  - ☒ Test coverage: ZkLighter.test.ts::L-937 (implicit — deposit registers account first)

### Negative behavior

- Revert when account index is invalid ( $>$  MAX\_ACCOUNT\_INDEX).
  - ☒ Negative test: ZkLighter.test.ts::L-910
- Revert when account index equals pool index.
  - ☐ Negative test: not tested
- Revert when pool index is  $\leq$  MAX\_MASTER\_ACCOUNT\_INDEX.
  - ☒ Negative test: ZkLighter.test.ts::L-916
- Revert when pool index is  $>$  MAX\_ACCOUNT\_INDEX.
  - ☒ Negative test: ZkLighter.test.ts::L-921
- Revert when share amount is  $<$  MIN\_POOL\_SHARES\_TO\_MINT\_OR\_BURN.

- ☒ Negative test: `ZkLighter.test.ts::L-926`
- Revert when share amount is `> MAX_POOL_SHARES_TO_MINT_OR_BURN`.
- ☒ Negative test: `ZkLighter.test.ts::L-932`
- Revert when caller is not registered.
- ☐ Negative test: not tested

## Function call analysis

- `getAccountIndexFromAddress(msg.sender)`
  - **What is controllable?** `msg.sender` is the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns master-account index from stored mapping. If `NIL_ACCOUNT_INDEX`, reverts. Cannot be manipulated — mapping is set during registration.
  - **What happens if it reverts, reenters or does other unusual control flow?** View function, no unusual control flow.
- `TxTypes.writeBurnSharesPubDataForPriorityQueue(_tx)`
  - **What is controllable?** All `_tx` struct parameters by caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns encoded bytes for priority queue. Malformed encoding could cause L2 share burning issues.
  - **What happens if it reverts, reenters or does other unusual control flow?** Library function, no revert conditions.
- `addPriorityRequest`
  - **What is controllable?** `pubData` derived from user inputs.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value — adds to priority queue. Queue corruption could prevent L2 from processing share burn.
  - **What happens if it reverts, reenters or does other unusual control flow?** Internal function, no revert conditions.

**Function:** `_deposit(address[] memory _to, uint16 _assetIndex, TxTypes.RouteType _routeType, uint256[] memory _amount)`

This is an internal function that implements the core deposit logic for both single and batch deposits. It handles ETH and ERC-20 transfers, validates amounts against tick sizes and caps, and handles fee-on-transfer tokens.

## Inputs

- `_to`
  - **Control:** User control (via `deposit()` and `depositBatch()`).
  - **Constraints:** Each address must not be `address(0)`. Array length must match `_amount`.
  - **Impact:** Recipient addresses for deposits.
- `_assetIndex`
  - **Control:** User control (via `deposit()` and `depositBatch()`).
  - **Constraints:** Must be valid (`NATIVE_ASSET_INDEX` or configured ERC-20). See `validateAssetIndex`.
  - **Impact:** Determines asset type and configuration.
- `_routeType`
  - **Control:** User control (via `deposit()` and `depositBatch()`).
  - **Constraints:** Type-safe enum (`RouteType`).
  - **Impact:** Routes deposit to perps or spot.
- `_amount`
  - **Control:** User control (via `deposit()` and `depositBatch()`).
  - **Constraints:** Each amount must be  $\geq \text{minDeposit}$ , aligned with `tickSize`, and sum must not exceed `depositCap`.
  - **Impact:** Amounts to deposit for each recipient.

## Branches and code coverage

### Intended branches

- Native asset (ETH) deposits should succeed when `msg.value` matches `totalAmount`.
  - ☑ Test coverage: `ZkLighter.t.sol:L-156 (test_deposit_eth_fail_and_success)`
- ERC-20 deposits should work with exact amounts.
  - ☑ Test coverage: `ZkLighter.t.sol:L-101 (test_deposit_usdc_fail_and_success)`
- Account should be auto-registered if depositing to unregistered address.
  - ☑ Test coverage: `ZkLighter.test.ts:L-701 (via registerDeposit)`
- Deposit should be added to priority queue.
  - ☑ Test coverage: `ZkLighter.test.ts:L-711 (NewPriorityRequest event, via registerDeposit)`

### Negative behavior

- Revert when non-native asset has no configured tokenAddress.
  - ☒ Negative test: ZkLighter.t.sol::L-110
- Revert when depositing ERC-20 with nonzero msg.value.
  - ☒ Negative test: ZkLighter.t.sol::L-126
- Revert when amount is < minDeposit or not aligned with tickSize.
  - ☒ Negative test: ZkLighter.t.sol::L-122
- Revert when recipient is address(0).
  - ☒ Negative test: ZkLighter.test.ts::L-686
- Revert when ETH msg.value != totalAmount.
  - ☒ Negative test: ZkLighter.t.sol::L-172
- Revert when ERC-20 transfer results in less balance than expected.
  - ☐ Negative test: not explicitly tested (fee-on-transfer tokens)
- Revert when balanceAfter exceeds depositCap.
  - ☒ Negative test: ZkLighter.test.ts::L-692

## Function call analysis

- IERC20(assetConfig.tokenAddress).balanceOf(address(this))
  - **What is controllable?** Nothing — reads contract's balance.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Used to calculate the actual received amount.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
External view call. If a token has a malicious balanceOf, it could cause issues but is protected by asset whitelisting.
- SafeERC20.safeTransferFrom(\_token, msg.sender, address(this), totalAmount)
  - **What is controllable?** Caller controls msg.sender and amount (validated).
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts on failure. Handles nonstandard ERC-20s.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Reverts entire transaction. Reentrancy protected at external function level.
- increaseBalanceToWithdraw(TREASURY\_ACCOUNT\_INDEX, \_assetIndex, baseExcessAmount)
  - **What is controllable?** baseExcessAmount is calculated from the actual received amount (excess from fee-on-transfer tokens).
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — updates pendingBalances mapping. If excess calculation

is wrong, treasury gets incorrect credit.

- **What happens if it reverts, reenters or does other unusual control flow?**  
Internal function, no reentrancy risk. Could overflow if balance exceeds uint128, but unlikely with validated inputs.
- `registerDeposit(_to[i], _assetIndex, _routeType, baseAmount)`
  - **What is controllable?** All parameters derived from validated inputs.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — adds deposit to priority queue for L2 processing. Data encoding is deterministic from validated inputs, so malformed data is unlikely.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Could revert if `lastAccountIndex` exceeds `MAX_ACCOUNT_INDEX` ( $2^{48} - 2$ ), but practically impossible to reach.

## Function: `validateCommonPerpMarketParams(TxTypes.CommonPerpsData memory perpParams)`

This is an internal pure function that validates common parameters for perps markets including fees, margin fractions, interest rates, funding clamps, and order limits.

### Inputs

- `perpParams`
  - **Control:** Called by `validateCreateMarketParams` and `validateUpdateMarketParams` with governor-provided data.
  - **Constraints:** Complex hierarchy of constraints. Fees are  $\leq$  `FEE_TICK`, margin fractions are in proper order (`closeOut` < `maintenance` < `minInitial` < `defaultInitial`  $\leq$  `MARGIN_TICK`), the interest rate is  $\leq$  `FUNDING_RATE_TICK`, funding clamps are  $\leq$  `FUNDING_RATE_TICK`, minimum amounts are in valid ranges, the order quote limit is  $\geq$  `minQuoteAmount`, and the open interest limit is  $\geq$  `orderQuoteLimit`.
  - **Impact:** Defines core economic parameters for perps market.

## Branches and code coverage

### Intended branches

- Fees (maker, taker, liquidation) should be validated as  $\leq$  `FEE_TICK`.
  - ☑ Test coverage: `ZkLighter.test.ts::L-943` (create) and `ZkLighter.test.ts::L-1270` (update)
- Margin fraction hierarchy should be enforced.
  - ☑ Test coverage: `ZkLighter.test.ts::L-943` (create) and

ZkLighter.test.ts::L-1270 (update)

- Interest rate should be  $\leq$  FUNDING\_RATE\_TICK.
  - ☒ Test coverage: ZkLighter.test.ts::L-943 (create) and ZkLighter.test.ts::L-1270 (update)
- Funding clamps should be  $\leq$  FUNDING\_RATE\_TICK.
  - ☐ Test coverage: not explicitly tested
- Minimum base/quote amounts should be  $> 0$  and within limits.
  - ☒ Test coverage: ZkLighter.test.ts::L-943 (create) and ZkLighter.test.ts::L-1270 (update)
- Order quote limit and open interest limit should be within valid ranges.
  - ☒ Test coverage: ZkLighter.test.ts::L-943 (create) and ZkLighter.test.ts::L-1270 (update)

### Negative behavior

- Revert when makerFee  $>$  FEE\_TICK.
  - ☒ Negative test: ZkLighter.test.ts::L-1008
- Revert when takerFee  $>$  FEE\_TICK.
  - ☒ Negative test: ZkLighter.test.ts::L-1016
- Revert when liquidationFee  $>$  FEE\_TICK.
  - ☒ Negative test: ZkLighter.test.ts::L-1025
- Revert when closeOutMarginFraction is 0.
  - ☒ Negative test: ZkLighter.test.ts::L-1037
- Revert when closeOut  $\geq$  maintenance.
  - ☒ Negative test: ZkLighter.test.ts::L-1044
- Revert when maintenance  $\geq$  minInitial.
  - ☒ Negative test: ZkLighter.test.ts::L-1051
- Revert when minInitial  $\geq$  defaultInitial.
  - ☒ Negative test: ZkLighter.test.ts::L-1059
- Revert when defaultInitial  $>$  MARGIN\_TICK.
  - ☒ Negative test: ZkLighter.test.ts::L-1068
- Revert when interestRate  $>$  FUNDING\_RATE\_TICK.
  - ☒ Negative test: ZkLighter.test.ts::L-1079
- Revert when funding clamps are invalid.
  - ☐ Negative test: not explicitly tested
- Revert when minBaseAmount is 0 or  $>$  MAX\_ORDER\_BASE\_AMOUNT.

- ☒ Negative test: `ZkLighter.test.ts::L-1102`
  - Revert when `minQuoteAmount` is 0 or `> MAX_ORDER_QUOTE_AMOUNT`.
- ☒ Negative test: `ZkLighter.test.ts::L-1109`
  - Revert when order quote limit is invalid.
- ☐ Negative test: not explicitly tested
  - Revert when open interest limit is invalid.
- ☐ Negative test: not explicitly tested

## Function call analysis

No external or internal function calls (pure validation logic).

## Function: `validateCommonSpotMarketParams(TxTypes.CommonSpotData memory spotParams)`

This is an internal pure function that validates common parameters for spot markets, including fees and order limits.

## Inputs

- `spotParams`
  - **Control:** Called by `validateCreateMarketParams` and `validateUpdateMarketParams` with governor-provided data.
  - **Constraints:** Fees are `<= FEE_TICK`, minimum amounts are in valid ranges, and order quote limit is `>= minQuoteAmount` and `<= MAX_ORDER_QUOTE_AMOUNT`.
  - **Impact:** Defines fee structure and order limits for the spot market.

## Branches and code coverage

### Intended branches

- Fees (maker, taker) should be validated as `<= FEE_TICK`.
  - ☒ Test coverage: `ZkLighter.test.ts::L-1120` (create) and `ZkLighter.test.ts::L-1407` (update)
- Minimum base/quote amounts should be `> 0` and within limits.
  - ☒ Test coverage: `ZkLighter.test.ts::L-1120` (create) and `ZkLighter.test.ts::L-1407` (update)
- Order quote limit should be within valid range.

- ☒ Test coverage: `ZkLighter.test.ts::L-1120` (create) and `ZkLighter.test.ts::L-1407` (update)

### Negative behavior

- Revert when `makerFee > FEE_TICK`.
  - ☒ Negative test: `ZkLighter.test.ts::L-1230`
- Revert when `takerFee > FEE_TICK`.
  - ☒ Negative test: `ZkLighter.test.ts::L-1237`
- Revert when `minBaseAmount` is 0 or `> MAX_ORDER_BASE_AMOUNT`.
  - ☒ Negative test: `ZkLighter.test.ts::L-1253`
- Revert when `minQuoteAmount` is 0 or `> MAX_ORDER_QUOTE_AMOUNT`.
  - ☒ Negative test: `ZkLighter.test.ts::L-1259`
- Revert when `orderQuoteLimit > MAX_ORDER_QUOTE_AMOUNT`.
  - ☐ Negative test: not explicitly tested
- Revert when `minQuoteAmount > orderQuoteLimit`.
  - ☒ Negative test: Implicit via minimum-amounts validation

### Function call analysis

No external or internal function calls (pure validation logic).

**Function:** `validateCreateMarketParams (TxTypes.CreateMarket _params)` `calldata`

This is an internal view function that performs comprehensive validation of market creation parameters.

### Inputs

- `_params (TxTypes.CreateMarket)`
  - **Control:** Called by `createMarket ( )` with governor-provided data.
  - **Type:** `TxTypes.CreateMarket` struct containing:
    - `marketIndex (uint16)`: Market identifier, must be within valid range for market type
    - `marketType (MarketType enum)`: Either Perps or Spot
    - `marketData (bytes)`: Packed data that gets decoded based on `marketType`

- For perps: Decoded into `CreateMarketPerpsData` containing `quoteMultiplier` (uint32) and common (`CommonPerpsData` with fees, margins, rates, limits)
- For spot: Decoded into `CreateMarketSpotData` containing `baseAssetIndex` (uint16), `quoteAssetIndex` (uint16), `sizeExtensionMultiplier` (uint56), `quoteExtensionMultiplier` (uint56), and common (`CommonSpotData` with fees, minimum amounts, order limits)
- **Constraints:** Validation depends on market type. For perps: market index must be  $\leq \text{MAX\_PERPS\_MARKET\_INDEX}$ , quote multiplier must be  $> 0$  and  $\leq \text{MAX\_QUOTE\_MULTIPLIER}$ , and common perps params (fees, margins, rates, limits) must be validated. For spot: market index must be in  $[\text{MIN\_SPOT\_MARKET\_INDEX}, \text{MAX\_SPOT\_MARKET\_INDEX}]$ , base and quote assets must be different and both must exist (validated via `validateAssetIndex`), extension multipliers must be  $> 0$ ,  $\leq \text{MAX\_ASSET\_EXTENSION\_MULTIPLIER}$ , and divisible by `FEE_TICK`, and common spot params (fees, minimum amounts, order limits) must be validated.
- **Impact:** Determines if market creation should proceed.

## Branches and code coverage

### Intended branches

- Perps market parameters should be fully validated including index, quote multiplier, and common params.
  - ☑ Test coverage: `test ZkLighter.test.ts:L-943`
- Spot market parameters should be fully validated including index range, asset indexes, extension multipliers, and common params.
  - ☑ Test coverage: `test ZkLighter.test.ts:L-1120`
- Base and quote assets must be different for spot markets.
  - ☑ Test coverage: Verified in `test ZkLighter.test.ts:L-1120` (creates market with different assets)
- Both assets must exist (validated via `validateAssetIndex`).
  - ☑ Test coverage: `ZkLighter.test.ts:L-1148`
- Extension multipliers must be greater than zero,  $\leq \text{MAX\_ASSET\_EXTENSION\_MULTIPLIER}$ , and divisible by `FEE_TICK`.
  - ☑ Test coverage: `ZkLighter.test.ts:L-1187` and `ZkLighter.test.ts:L-1207`

### Negative behavior

- Revert when the perps market index is  $> \text{MAX\_PERPS\_MARKET\_INDEX}$ .

- ☒ Negative test: `ZkLighter.test.ts::L-966`
  - Revert when the perps quote multiplier is 0 or > `MAX_QUOTE_MULTIPLIER`.
- ☒ Negative test: `ZkLighter.test.ts::L-986`
  - Revert when the spot market index is < `MIN_SPOT_MARKET_INDEX`.
- ☒ Negative test: `ZkLighter.test.ts::L-1167`
  - Revert when the spot market index is > `MAX_SPOT_MARKET_INDEX`.
- ☒ Negative test: `ZkLighter.test.ts::L-1177`
  - Revert when the spot base and quote assets are the same.
- ☒ Negative test: `ZkLighter.test.ts::L-1137`
  - Revert when the base or quote asset index is invalid.
- ☒ Negative test: `ZkLighter.test.ts::L-1148`
  - Revert when the size extension multiplier is 0, > `MAX_ASSET_EXTENSION_MULTIPLIER`, or not divisible by `FEE_TICK`.
- ☒ Negative test: `ZkLighter.test.ts::L-1207`
  - Revert when the quote extension multiplier is 0, > `MAX_ASSET_EXTENSION_MULTIPLIER`, or not divisible by `FEE_TICK`.
- ☒ Negative test: `ZkLighter.test.ts::L-1191`
  - Revert when the market type is invalid.
- ☐ Negative test (not explicitly tested)
  - Revert for any common param validation failures (delegated to validation functions).
- ☒ Negative test: Extensively tested (fees, margins, rates, etc.)

## Function call analysis

- `TxTypes.readCreateMarketPerpsData(_params.marketData)`
  - **What is controllable?** `marketData` bytes by the governor.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Decodes packed bytes into `CreateMarketPerpsData` struct containing `quoteMultiplier` and `common` (`CommonPerpsData`). The decoded values are then used for validation checks. If bytes length is incorrect, function reverts with "Invalid packed create market perps data length". Successfully decoded but invalid values are caught by subsequent validation. Pure library function, deterministic decoding.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Reverts if `_data.length != PACKED_CREATE_MARKET_PERPS_BYTES`.
- `TxTypes.readCreateMarketSpotData(_params.marketData)`
  - **What is controllable?** `marketData` bytes by the governor.



## Inputs

- `_params`
  - **Control:** Called by `updateMarket()` with governor-provided data.
  - **Constraints:** Market index must be valid for type. Status must be `ACTIVE` or `NONE`. All other params validated are the same as `createMarket`.
  - **Impact:** Determines if market update should proceed.

## Branches and code coverage

### Intended branches

- Perps-market updates should validate index, status, and common params.
  - ☒ Test coverage: `ZkLighter.test.ts:L-1270` (perps update tests)
- Spot-market updates should validate index range, status, and common params.
  - ☒ Test coverage: `ZkLighter.test.ts:L-1407` (spot update tests)

### Negative behavior

- Revert when perps-market index is invalid.
  - ☒ Negative test: `ZkLighter.test.ts:L-1291`
- Revert when perps status is not `ACTIVE` or `NONE`.
  - ☒ Negative test: `ZkLighter.test.ts:L-1307`
- Revert when spot-market index is out of range.
  - ☒ Negative test: `ZkLighter.test.ts:L-1419`
- Revert when spot status is not `ACTIVE` or `NONE`.
  - ☒ Negative test: `ZkLighter.test.ts:L-1435`
- Revert when market type is invalid.
  - ☐ Negative test: not explicitly tested
- Revert for invalid fees.
  - ☒ Negative test: `ZkLighter.test.ts:L-1315`
- Revert for invalid margin requirements.
  - ☒ Negative test: `ZkLighter.test.ts:L-1337`
- Revert for invalid interest rate.
  - ☒ Negative test: `ZkLighter.test.ts:L-1372`
- Revert for invalid min amounts.
  - ☒ Negative test: `ZkLighter.test.ts:L-1385`

## Function call analysis

Decodes market data via `TxTypes.readUpdateMarketPerpsData` or `TxTypes.readUpdateMarketSpotData`, then delegates to `validateCommonPerpMarketParams` or `validateCommonSpotMarketParams`. All pure functions.

### Function: `registerDeposit(address _toAddress, uint16 _assetIndex, TxTypes.RouteType _routeType, uint64 _baseAmount)`

This is an internal function that handles account registration (if needed) and adds a deposit to priority queue. It is critical for managing account indexes and the L1 → L2 deposit flow.

## Inputs

- `_toAddress`
  - **Control:** User control (provided via `_deposit`).
  - **Constraints:** If the address maps to a system account (`index <= MAX_SYSTEM_ACCOUNT_INDEX`), it is reset to address (0) within this function. If the address is not registered (`NIL_ACCOUNT_INDEX`), a new account is auto-registered up to the `MAX_MASTER_ACCOUNT_INDEX` limit.
  - **Impact:** Recipient address for deposit — may trigger account registration or address reset for system accounts.
- `_assetIndex`
  - **Control:** User control (provided via `_deposit`).
  - **Constraints:** Already validated in `_deposit` to ensure the asset exists (either `NATIVE_ASSET_INDEX` or a registered token asset). See `_deposit` for detailed validation.
  - **Impact:** Asset being deposited.
- `_routeType`
  - **Control:** User control (provided via `_deposit`).
  - **Constraints:** Type-safe enum (`TxTypes.RouteType`) with values `Perps` (0) or `Spot` (1). No explicit runtime validation in `_deposit`, but enum type prevents invalid values at compile time.
  - **Impact:** Routing of deposit on L2 (perps or spot market).
- `_baseAmount`
  - **Control:** Derived from user-provided amounts (calculated by `_deposit`).
  - **Constraints:** Converted from token amount to base units. Original amount is validated for minimum deposit, divisibility by tick size, and deposit cap limits in `_deposit`.
  - **Impact:** Amount deposited in base units (uint64).

## Branches and code coverage

### Intended branches

- Existing accounts should use their existing account index.
  - ☒ Test coverage: test ZkLighter.test.ts::L-252
- System accounts (index <= MAX\_SYSTEM\_ACCOUNT\_INDEX) should have the address set to address(0).
  - ☒ Test coverage: test ZkLighter.test.ts::L-710 (treasury) and ZkLighter.test.ts::L-715 (insurance fund)
- New accounts (NIL\_ACCOUNT\_INDEX) should autoregister with incremented lastAccountIndex.
  - ☒ Test coverage: test ZkLighter.test.ts::L-252 (first-time deposit)
- A priority request should be added with correct deposit data.
  - ☒ Test coverage: Verified in test ZkLighter.test.ts::L-252 via NewPriorityRequest event

### Negative behavior

- None (validation in \_deposit).

## Function call analysis

- `getAccountIndexFromAddress(_toAddress)`
  - **What is controllable?** `_toAddress` by the user.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the stored mapping value from `addressToAccountIndex`. Returns `NIL_ACCOUNT_INDEX` if address is not registered. The return value directly controls critical logic: if <= `MAX_SYSTEM_ACCOUNT_INDEX`, address is reset to `address(0)`; if `NIL_ACCOUNT_INDEX`, triggers auto-registration; otherwise uses existing index. If the mapping contains unexpected values (e.g., corrupted state), it could cause incorrect account handling, wrong address assignments, or unintended account registration. Additionally, if a user provides an address that maps to a system account index, the deposit address will be reset to `address(0)`, which may not be the intended behavior.
  - **What happens if it reverts, reenters or does other unusual control flow?** View function, no unusual control flow.
- `TxTypes.writeDepositPubDataForPriorityQueue(_tx)`
  - **What is controllable?** All transaction parameters are derived from validated inputs and internal state.
  - **If the return value is controllable, how is it used and how can it go wrong?** Encodes deposit data into packed bytes for the priority queue. The encoded

bytes are then passed to `addPriorityRequest` for L2 processing. Pure library function, deterministic encoding.

- **What happens if it reverts, reenters or does other unusual control flow?**

Pure function, no revert conditions.

- `addPriorityRequest(TxTypes.PriorityPubDataTypeL1Deposit, pubData, pubData)`

- **What is controllable?** `pubData` derived from validated inputs.

- **If the return value is controllable, how is it used and how can it go wrong?**

Adds the deposit to the priority request queue for L2 processing. See `addPriorityRequest` for detailed analysis.

- **What happens if it reverts, reenters or does other unusual control flow?** No revert conditions.

### Function: `addPriorityRequest(uint8 _pubdataType, bytes memory _priorityRequest, bytes memory _pubDataWithMetadata)`

This is a critical internal function that adds requests to the priority queue for L2 processing. It creates a cryptographic hash chain (`prefixHash`) to ensure request ordering and integrity. It sets expiration timestamps for requests.

### Inputs

- `_pubdataType`
  - **Control:** Called by various functions with fixed type constants.
  - **Constraints:** Should be one of the predefined `PriorityPubDataType` constants.
  - **Impact:** Identifies the type of priority operation (deposit, market creation, order creation, etc.).
- `_priorityRequest`
  - **Control:** Encoded data from calling functions.
  - **Constraints:** Padded to `MAX_PRIORITY_REQUEST_PUBDATA_SIZE` bytes. Used for hash-chain creation.
  - **Impact:** The actual request data hashed and stored.
- `_pubDataWithMetadata`
  - **Control:** Encoded data from calling functions — may include additional metadata.
  - **Constraints:** Only used for event emission, not stored.
  - **Impact:** Request data emitted in event for off-chain processing.

## Branches and code coverage

### Intended branches

- Priority request should be added with correct hash chain and expiration.
  - ☑ Test coverage: Implicit via all priority-request operations (e.g., `ZkLighter.test.ts:L-95`)
- PrefixHash should be Keccak-256 of (previous prefixHash || padded pubdata).
  - ☑ Test coverage: `ZkLighter.t.sol:L-502` (hash validation in desert mode test)
- NewPriorityRequest event should be emitted.
  - ☑ Test coverage: `ZkLighter.test.ts:L-95, L-119, L-140, etc.`
- openPriorityRequestCount should be incremented.
  - ☑ Test coverage: `ZkLighter.test.ts:L-1479 and L-1513`

### Negative behavior

- None (internal function with validated inputs from calling functions).

## Function call analysis

Minimal external calls — only `SafeCast.toUint64` for safe timestamp conversion. Rest is direct storage updates, hash computation, and event emission.

### Function: `increaseBalanceToWithdraw(uint48 _masterAccountIndex, uint16 _assetIndex, uint128 _baseAmount)`

This is an internal function that increases the withdrawable balance for an account. It is called by `_deposit` when handling excess tokens from ERC-20 transfers (excess goes to treasury).

### Inputs

- `_masterAccountIndex`
  - **Control:** Called by `_deposit` with `TREASURY_ACCOUNT_INDEX` for excess amounts.
  - **Constraints:** Should be a valid master-account index.
  - **Impact:** The account whose withdrawable balance increases.
- `_assetIndex`
  - **Control:** From `_deposit` — already validated.
  - **Constraints:** Should be a valid asset index.

- **Impact:** The asset whose balance increases.
- `_baseAmount`
  - **Control:** Calculated by `_deposit` from excess token amount.
  - **Constraints:** Must fit in `uint128`. Added to existing balance.
  - **Impact:** Amount to add to withdrawable balance.

## Branches and code coverage

### Intended branches

- Balance should be increased by the specified amount.
- ☒ Test coverage: Identical implementation in `ZkLighter.sol` tested via `executeBatches` (`ZkLighter.t.sol::L-627`, assertion at `L-636`)

### Negative behavior

- None (internal function with validated inputs from `_deposit`).

## Function call analysis

No external or internal function calls (direct storage update).

### Function: `validateAssetIndex(uint16 _assetIndex)`

This is a simple internal view function that validates that an asset index exists in the system configuration. It is used to ensure operations only reference configured assets.

### Inputs

- `_assetIndex`
  - **Control:** From calling functions (typically governor or validated user input).
  - **Constraints:** Must be `NATIVE_ASSET_INDEX` or have a configured `tokenAddress`.
  - **Impact:** Asset being validated for existence.

## Branches and code coverage

### Intended branches

- `NATIVE_ASSET_INDEX` should always be considered valid.
- ☒ Test coverage: Implicit via `depositNative` calls (e.g.,

ZkLighter.test.ts:L-723)

- Non-native assets should be validated by checking `tokenAddress != address(0)`.

- ☑ Test coverage: Implicit via `depositUSDC` calls (e.g., ZkLighter.test.ts:L-698)

### Negative behavior

- Revert when non-native asset has `tokenAddress == address(0)`.
- ☑ Negative test: ZkLighter.test.ts:L-798 (invalid asset in withdraw) and ZkLighter.test.ts:L-1151 (invalid base asset in spot market) and ZkLighter.test.ts:L-1160 (invalid quote asset in spot market)

### Function call analysis

No external or internal function calls (simple storage read and comparison).

## 5.2. Module: ZkLighter.sol

### Function: `initialize(bytes calldata initializationParameters)`

This is a contract initialization function that sets up governance, verifiers, and genesis state roots and registers default asset configurations (ETH and USDC). It can only be called once on a proxy, not on the implementation.

### Inputs

- `initializationParameters`
  - **Control:** Deployer control during deployment.
  - **Constraints:** Must decode to (governanceAddress, verifierAddress, additionalZkLighter, desertVerifier, genesisStateRoot, genesisValidiumRoot). All addresses must have code deployed.
  - **Impact:** Initializes critical contract dependencies and genesis state.

### Branches and code coverage

#### Intended branches

- Should decode all six parameters correctly from `initializationParameters`.
- ☑ Test coverage: Proxy deployment in `Proxy.test.ts` (beforeEach lines 57-70) successfully initializes
- Should set verifier, governance, `additionalZkLighter`, and `desertVerifier` correctly.

- ☒ Test coverage: Implicit — all tests depend on these being set correctly
- Should create and store genesis batch (batch 0) with provided state roots.
- ☒ Test coverage: Tests use genesis batch (storedBatchInfo with batchNumber 1)
- Should call `_registerDefaultAssetConfigs` to set up ETH and USDC.
- ☒ Test coverage: `test_check_default_asset_configs()` lines 284–314 verify ETH and USDC configs

#### Negative behavior

- Revert when called on implementation instead of proxy (`ZkLighter_CannotBeInitialisedByImpl`).
- ☐ Negative test (proxy pattern not explicitly tested)
- Revert when any address does not have code (`ZkLighter_InvalidInitializeParameters`).
- ☐ Negative test (not explicitly tested)
- Revert when called more than once (initializer modifier).
- ☐ Negative test (not explicitly tested)

#### Function call analysis

- `_registerDefaultAssetConfigs()`
  - **What is controllable?** Nothing, hardcoded ETH and USDC configs.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value. Sets up default assets.
  - **What happens if it reverts, reenters or does other unusual control flow?** Could revert if the USDC address from governance is invalid.

#### Function: `upgrade(bytes calldata upgradeParameters)`

This is a contract upgrade function that allows updating `additionalZkLighter`, `desertVerifier`, and `stateRootUpgradeVerifier` addresses. It uses a commitment hash to ensure upgrade parameters are known beforehand.

#### Inputs

- `upgradeParameters`
  - **Control:** Caller control (but protected by commitment check).

- **Constraints:** Must hash to hardcoded `initializationParametersCommitment` (0x46700b4d...). Must decode to (additionalZkLighter, desertVerifier, stateRootUpgradeVerifier). Nonzero addresses must have code.
- **Impact:** Updates critical contract dependencies.

## Branches and code coverage

### Intended branches

- Should only be callable via `Proxy.upgradeTarget()`.
  - ☒ Test coverage: `Proxy.test.ts:L-165`, `Proxy.test.ts:L-195`
- Should validate upgrade parameters hash matches commitment and update contract addresses.
  - ☐ Test coverage: not tested (upgrade flow via proxy)

### Negative behavior

- Revert when called on implementation.
  - ☒ Negative test: `Proxy.test.ts:L-195`
- Revert when parameters hash mismatches or address has no code.
  - ☐ Negative test: not tested

## Function call analysis

- `keccak256(upgradeParameters)`
  - **What is controllable?** `upgradeParameters` by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Compared against the hardcoded commitment. Prevents arbitrary upgrades.
  - **What happens if it reverts, reenters or does other unusual control flow?** Deterministic hash, no revert.
- `abi.decode(upgradeParameters, (address, address, address))`
  - **What is controllable?** `upgradeParameters` (but must match commitment).
  - **If the return value is controllable, how is it used and how can it go wrong?** Used to update contract references. Protected by commitment and code checks.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts if decode fails. Protected by `nonReentrant`.
- `_hasCode(_additionalZkLighter)` (and others)
  - **What is controllable?** Addresses from parameters.

- **If the return value is controllable, how is it used and how can it go wrong?**  
Must return true or revert.
- **What happens if it reverts, reenters or does other unusual control flow?**  
Pure check, no unusual flow.

### Function: `_registerDefaultAssetConfigs()`

This is an internal function called during initialization to register default asset configurations for ETH (native asset) and USDC. It sets up hardcoded parameters for these two core assets.

## Branches and code coverage

### Intended branches

- ETH config should be registered at `NATIVE_ASSET_INDEX` with correct parameters.  
☒ Test coverage: `ZkLighter.t.sol:L-295 to L-303`
- USDC config should be registered at `USDC_ASSET_INDEX` with address from `governance.usdc()`.  
☒ Test coverage: `ZkLighter.t.sol:L-285 to L-293`

### Negative behavior

- Could revert if the `governance.usdc()` call fails.  
☐ Negative test (not tested — would require faulty governance contract)

## Function call analysis

- `governance.usdc()`
  - **What is controllable?** Nothing, reads USDC address from the governance contract.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Used to set the USDC token address. Must be set correctly in the governance contract.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
External view call — could revert if not set. Called during initialization.

**Function: registerAssetConfig(uint16 assetIndex, address tokenAddress, uint8 withdrawalsEnabled, uint56 extensionMultiplier, uint128 tickSize, uint64 depositCapTicks, uint64 minDepositTicks)**

This is a governor-only function that registers new asset configurations on L1. This is separate from the L2 asset registration and sets up the L1-side asset parameters.

## Inputs

- assetIndex
  - **Control:** Governor control.
  - **Constraints:** Must be in [MIN\_ASSET\_INDEX, MAX\_ASSET\_INDEX]. Cannot be NATIVE\_ASSET\_INDEX. Must not be already registered.
  - **Impact:** Index for the new asset.
- tokenAddress
  - **Control:** Governor control.
  - **Constraints:** Cannot be address(0). Must not be already mapped. Must have code deployed.
  - **Impact:** ERC-20 token contract address.
- withdrawalsEnabled
  - **Control:** Governor control.
  - **Constraints:** Must be 0 or 1.
  - **Impact:** Whether withdrawals are enabled for this asset.
- extensionMultiplier
  - **Control:** Governor control.
  - **Constraints:** Must be greater than zero and  $\leq$  MAX\_ASSET\_EXTENSION\_MULTIPLIER. Extended cap (multiplier \* depositCapTicks) must be  $\leq$  MAX\_EXTENDED\_DEPOSIT\_CAP\_TICKS.
  - **Impact:** Multiplier for extended deposit capacity.
- tickSize
  - **Control:** Governor control.
  - **Constraints:** Must be greater than zero and  $\leq$  MAX\_TICK\_SIZE.
  - **Impact:** Smallest unit for this asset.
- depositCapTicks
  - **Control:** Governor control.
  - **Constraints:** Must be greater than zero and  $\leq$  MAX\_DEPOSIT\_CAP\_TICKS.
  - **Impact:** Maximum deposits allowed in ticks.
- minDepositTicks

- **Control:** Governor control.
- **Constraints:** Must be greater than zero and  $\leq$  depositCapTicks.
- **Impact:** Minimum deposit required in ticks.

## Branches and code coverage

### Intended branches

- Should register new ERC-20 asset config with all parameters.
- ☒ Test coverage: ZkLighter.t.sol::L-97 (LIT in setUp()), ZkLighter.t.sol::L-305

### Negative behavior

- Revert when caller is not governor or parameters are invalid.
- ☐ Negative test: not tested

## Function call analysis

- governance.requireGovernor(msg.sender)
  - **What is controllable?** msg.sender by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if not governor.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Prevents unauthorized registration.
- \_hasCode(tokenAddress)
  - **What is controllable?** tokenAddress by the governor.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Must return true or revert.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Pure check, no unusual flow.

### Function: updateAssetConfig(uint16 assetIndex, uint8 withdrawalsEnabled, uint64 depositCapTicks, uint64 minDepositTicks)

This is a governor-only function to update existing asset-configuration parameters. It cannot disable withdrawals once enabled (one-way transition). Cannot change token address, extension multiplier, or tick size.

## Inputs

- `assetIndex`
  - **Control:** Governor control.
  - **Constraints:** Must be valid (`NATIVE_ASSET_INDEX` or configured asset).
  - **Impact:** Asset to update.
- `withdrawalsEnabled`
  - **Control:** Governor control.
  - **Constraints:** Must be 0 or 1. Cannot go from 1 to 0 (withdrawals can only be enabled, not disabled).
  - **Impact:** Withdrawal status for asset.
- `depositCapTicks`
  - **Control:** Governor control.
  - **Constraints:** Must be greater than zero and  $\leq$  `MAX_DEPOSIT_CAP_TICKS`. Extended cap must be  $\leq$  `MAX_EXTENDED_DEPOSIT_CAP_TICKS`.
  - **Impact:** Maximum deposits allowed in ticks.
- `minDepositTicks`
  - **Control:** Governor control.
  - **Constraints:** Must be greater than zero and  $\leq$  `depositCapTicks`.
  - **Impact:** Minimum deposit required in ticks.

## Branches and code coverage

### Intended branches

- Should allow governor to update config and enable withdrawals (0  $\rightarrow$  1).
  - ☒ Test coverage: `ZkLighter.t.sol::L-260 to L-273`

### Negative behavior

- Revert when caller is not governor, asset invalid, or trying to disable withdrawals.
  - ☐ Negative test: not tested

## Function call analysis

- `governance.requireGovernor(msg.sender)`
  - **What is controllable?** `msg.sender` by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if not governor.

- **What happens if it reverts, reenters or does other unusual control flow?**  
Prevents unauthorized updates.

**Function: registerAsset(uint8 \_11Decimals, uint8 \_decimals, bytes32 \_symbol, TxTypes.RegisterAsset calldata \_params)**

External wrapper that delegates to `AdditionalZkLighter.registerAsset`. All logic and validations are in the delegated contract.

## Inputs

All inputs passed through to `AdditionalZkLighter.registerAsset` — see that threat model for details.

## Branches and code coverage

### Intended branches

- Should delegate to `AdditionalZkLighter`.  
☒ Test coverage: see `AdditionalZkLighter.registerAsset`

### Negative behavior

- All validation in delegated contract.  
☒ Negative test: see `AdditionalZkLighter.registerAsset`

## Function call analysis

- `delegateAdditional`
  - **What is controllable?** All calldata passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
`delegatecall` preserves `msg.sender` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Reverts bubble up. Reentrancy protection in delegated function.

**Function: updateAsset(TxTypes.UpdateAsset calldata \_params)**

External wrapper that delegates to `AdditionalZkLighter.updateAsset`. All logic and validations are in the delegated contract.

## Inputs

All inputs passed through to `AdditionalZkLighter.updateAsset` — see that threat model for details.

## Branches and code coverage

### Intended branches

- Should delegate to `AdditionalZkLighter`.
  - ☑ Test coverage: see `AdditionalZkLighter.updateAsset`

### Negative behavior

- All validation in delegated contract.
  - ☑ Negative test: see `AdditionalZkLighter.updateAsset`

## Function call analysis

- `delegateAdditional`
  - **What is controllable?** All calldata passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?** `delegatecall` preserves `msg.sender` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts bubble up. Reentrancy protection in delegated function.

## Function: `deposit(address _to, uint16 _assetIndex, TxTypes.RouteType _routeType, uint256 _amount)`

External payable wrapper that delegates to `AdditionalZkLighter.deposit`. Supports both ETH and ERC-20 deposits. All logic and validations are in the delegated contract.

## Inputs

All inputs passed through to `AdditionalZkLighter.deposit` — see that threat model for details. `msg.value` also preserved for ETH deposits.

## Branches and code coverage

### Intended branches

- Should delegate to `AdditionalZkLighter`.

- ☑ Test coverage: see `AdditionalZkLighter.deposit`

### Negative behavior

- All validation in delegated contract.

- ☑ Negative test: see `AdditionalZkLighter.deposit`

### Function call analysis

- `delegateAdditional`
  - **What is controllable?** All `calldata` and `msg.value` passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?** `delegatecall` preserves `msg.sender` and `msg.value` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts bubble up. Reentrancy protection in delegated function.

**Function:** `depositBatch(uint64[] calldata _amount, address[] calldata _to, uint48[] calldata _accountIndex)`

External wrapper that delegates to `AdditionalZkLighter.depositBatch`. Supports batch USDC deposits. All logic and validations are in the delegated contract.

### Inputs

All inputs passed through to `AdditionalZkLighter.depositBatch` — see that threat model for details.

### Branches and code coverage

#### Intended branches

- Should delegate to `AdditionalZkLighter`.

- ☑ Test coverage: see `AdditionalZkLighter.depositBatch`

#### Negative behavior

- All validation in delegated contract.

- ☑ Negative test: see `AdditionalZkLighter.depositBatch`

## Function call analysis

- `delegateAdditional`
  - **What is controllable?** All calldata passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?** `delegatecall` preserves `msg.sender` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts bubble up. Reentrancy protection in delegated function.

### Function: `changePubKey(uint48 _accountIndex, uint8 _apiKeyIndex, bytes calldata _pubKey)`

External wrapper that delegates to `AdditionalZkLighter.changePubKey`. All logic and validations (including Goldilocks field checks) are in the delegated contract.

## Inputs

All inputs passed through to `AdditionalZkLighter.changePubKey` — see that threat model for details.

## Branches and code coverage

### Intended branches

- Should delegate to `AdditionalZkLighter`.
  - ☒ Test coverage: see `AdditionalZkLighter.changePubKey`

### Negative behavior

- All validation in delegated contract.
  - ☒ Negative test: see `AdditionalZkLighter.changePubKey`

## Function call analysis

- `delegateAdditional`
  - **What is controllable?** All calldata passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?** `delegatecall` preserves `msg.sender` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts bubble up. Reentrancy protection in delegated function.

**Function:** `createMarket(uint8 _size_decimals, uint8 _price_decimals, bytes32 _symbol, TxTypes.CreateMarket calldata _params)`

External wrapper that delegates to `AdditionalZkLighter.createMarket`. Governor-only. All logic and validations are in the delegated contract.

## Inputs

All inputs passed through to `AdditionalZkLighter.createMarket` — see that threat model for details.

## Branches and code coverage

### Intended branches

- Should delegate to `AdditionalZkLighter`.
  - ☑ Test coverage: see `AdditionalZkLighter.createMarket`

### Negative behavior

- All validation in delegated contract.
  - ☑ Negative test: see `AdditionalZkLighter.createMarket`

## Function call analysis

- `delegateAdditional`
  - **What is controllable?** All calldata passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?** `delegatecall` preserves `msg.sender` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts bubble up. Reentrancy protection in delegated function.

**Function:** `updateMarket(TxTypes.UpdateMarket calldata _params)`

External wrapper that delegates to `AdditionalZkLighter.updateMarket`. Governor-only. All logic and validations are in the delegated contract.

## Inputs

All inputs passed through to `AdditionalZkLighter.updateMarket` — see that threat model for details.

## Branches and code coverage

### Intended branches

- Should delegate to `AdditionalZkLighter`.
  - ☑ Test coverage: see `AdditionalZkLighter.updateMarket`

### Negative behavior

- All validation in delegated contract.
  - ☑ Negative test: see `AdditionalZkLighter.updateMarket`

## Function call analysis

- `delegateAdditional`
  - **What is controllable?** All calldata passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
`delegatecall` preserves `msg.sender` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Reverts bubble up. Reentrancy protection in delegated function.

## Function: `cancelAllOrders(uint48 _accountIndex)`

External wrapper that delegates to `AdditionalZkLighter.cancelAllOrders`. All logic and validations are in the delegated contract.

### Inputs

All inputs passed through to `AdditionalZkLighter.cancelAllOrders` — see that threat model for details.

## Branches and code coverage

### Intended branches

- Should delegate to `AdditionalZkLighter`.
  - ☑ Test coverage: see `AdditionalZkLighter.cancelAllOrders`

### Negative behavior

- All validation in delegated contract.
  - ☑ Negative test: see `AdditionalZkLighter.cancelAllOrders`

## Function call analysis

- `delegateAdditional`
  - **What is controllable?** All calldata passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?** `delegatecall` preserves `msg.sender` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts bubble up. Reentrancy protection in delegated function.

**Function:** `withdraw(uint48 _accountIndex, uint16 _assetIndex, Tx-Types.RouteType _routeType, uint64 _baseAmount)`

External wrapper that delegates to `AdditionalZkLighter.withdraw`. Creates a priority request for L2 withdrawal. All logic and validations are in the delegated contract.

## Inputs

All inputs passed through to `AdditionalZkLighter.withdraw` — see that threat model for details.

## Branches and code coverage

### Intended branches

- Should delegate to `AdditionalZkLighter`.
  - ☒ Test coverage: see `AdditionalZkLighter.withdraw`

### Negative behavior

- All validation in delegated contract.
  - ☒ Negative test: see `AdditionalZkLighter.withdraw`

## Function call analysis

- `delegateAdditional`
  - **What is controllable?** All calldata passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?** `delegatecall` preserves `msg.sender` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts bubble up. Reentrancy protection in delegated function.

**Function: `createOrder(uint48 _accountIndex, uint16 _marketIndex, uint48 _baseAmount, uint32 _price, uint8 _isAsk, uint8 _orderType)`**

External wrapper that delegates to `AdditionalZkLighter.createOrder`. Creates limit or market orders. All logic and validations are in the delegated contract.

## Inputs

All inputs passed through to `AdditionalZkLighter.createOrder` — see that threat model for details.

## Branches and code coverage

### Intended branches

- Should delegate to `AdditionalZkLighter`.
- ☑ Test coverage: see `AdditionalZkLighter.createOrder`

### Negative behavior

- All validation in delegated contract.
- ☑ Negative test: see `AdditionalZkLighter.createOrder`

## Function call analysis

- `delegateAdditional`
  - **What is controllable?** All calldata passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?** `delegatecall` preserves `msg.sender` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts bubble up. Reentrancy protection in delegated function.

**Function: `burnShares(uint48 _accountIndex, uint48 _publicPoolIndex, uint64 _shareAmount)`**

External wrapper that delegates to `AdditionalZkLighter.burnShares`. Burns shares in a public pool. All logic and validations are in the delegated contract.

## Inputs

All inputs passed through to `AdditionalZkLighter.burnShares` — see that threat model for details.

## Branches and code coverage

### Intended branches

- Should delegate to `AdditionalZkLighter`.
  - ☑ Test coverage: see `AdditionalZkLighter.burnShares`

### Negative behavior

- All validation in delegated contract.
  - ☑ Negative test: see `AdditionalZkLighter.burnShares`

## Function call analysis

- `delegateAdditional`
  - **What is controllable?** All calldata passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?** `delegatecall` preserves `msg.sender` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts bubble up. Reentrancy protection in delegated function.

**Function:** `updateStateRoot(StoredBatchInfo calldata _lastStoredBatch, bytes32 _stateRoot, bytes32 _validiumRoot, bytes calldata proof)`

External wrapper that delegates to `AdditionalZkLighter.updateStateRoot`. Updates state root with ZK proof verification. All logic and validations are in the delegated contract.

## Inputs

All inputs passed through to `AdditionalZkLighter.updateStateRoot` — see that threat model for details.

## Branches and code coverage

### Intended branches

- Should delegate to `AdditionalZkLighter`.

- ☑ Test coverage: see `AdditionalZkLighter.updateStateRoot`

### Negative behavior

- All validation in delegated contract.
- ☑ Negative test: see `AdditionalZkLighter.updateStateRoot`

### Function call analysis

- `delegateAdditional`
  - **What is controllable?** All calldata passed through.
  - **If the return value is controllable, how is it used and how can it go wrong?** `delegatecall` preserves `msg.sender` context. Reverts propagate correctly.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts bubble up. Reentrancy protection in delegated function.

**Function:** `createExitCommitment(uint256 stateRoot, uint48 _accountIndex, uint48 _masterAccountIndex, uint16 _assetIndex, uint128 _totalBaseAmount)`

This is an internal pure function that creates a Keccak-256 commitment hash for desert-mode exits. It is used by `performDesert` to verify that a user's exit parameters match the ZK-proven state.

### Inputs

- `stateRoot`
  - **Control:** From calling function (`performDesert`).
  - **Constraints:** Should be the last verified state root.
  - **Impact:** State root included in commitment.
- `_accountIndex`
  - **Control:** From calling function (caller-provided).
  - **Constraints:** L2 account index.
  - **Impact:** Account index included in commitment.
- `_masterAccountIndex`
  - **Control:** From calling function (caller-provided).
  - **Constraints:** L1 master-account index.
  - **Impact:** Master-account index included in commitment.
- `_assetIndex`
  - **Control:** From calling function (caller-provided).

- **Constraints:** Asset index for withdrawal.
  - **Impact:** Asset included in commitment.
- `_totalBaseAmount`
  - **Control:** From calling function (caller-provided, ZK proven).
  - **Constraints:** Must match proven balance.
  - **Impact:** Amount included in commitment.

## Branches and code coverage

### Intended branches

- Should create deterministic Keccak-256 hash of packed parameters for ZK proof verification.
- ☐ Test coverage: not tested

### Negative behavior

- Pure function, no revert conditions.

## Function call analysis

Pure function using `keccak256(abi.encodePacked(...))`. No external calls.

**Function: `performDesert(uint48 _accountIndex, uint48 _masterAccountIndex, uint16 _assetIndex, uint128 _totalBaseAmount, bytes calldata proof)`**

This function allows users to exit ZkLighter in desert mode by providing a ZK proof of their account balance. Desert mode is activated when priority requests expire without processing.

## Inputs

- `_accountIndex`
  - **Control:** User control.
  - **Constraints:** Must not have already performed desert for this asset. The proof must verify this account index.
  - **Impact:** Account exiting in desert mode.
- `_masterAccountIndex`
  - **Control:** User control.
  - **Constraints:** Must match the master account that owns `_accountIndex`

- (verified by proof).
- **Impact:** Master account to receive withdrawn balance.
- `_assetIndex`
  - **Control:** User control.
  - **Constraints:** Must be valid asset. Account must not have already exited for this asset.
  - **Impact:** Asset being withdrawn.
- `_totalBaseAmount`
  - **Control:** User control (but must be proven).
  - **Constraints:** Must be provable via ZK proof against current `stateRoot`.
  - **Impact:** Amount to withdraw in base units.
- `proof`
  - **Control:** User control.
  - **Constraints:** Must be a valid ZK proof verifying account balance for this asset.
  - **Impact:** Cryptographic proof of balance.

## Branches and code coverage

### Intended branches

- Desert mode must be active and account must not have already exited for this asset.
  - ☒ Test coverage: `ZkLighter.t.sol:L-317, L-413 to L-421`
- ZK proof should be verified and balance credited to master account.
  - ☒ Test coverage: `ZkLighter.t.sol:L-333 to L-342`

### Negative behavior

- Revert when desert mode inactive, already performed, or proof fails.
  - ☒ Negative test: `ZkLighter.t.sol:L-344 to L-353, L-406 to L-421`

## Function call analysis

- `createExitCommitment(uint256(stateRoot), _accountIndex, _masterAccountIndex, _assetIndex, _totalBaseAmount)`
  - **What is controllable?** All parameters except `stateRoot` (which is a contract state).
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the deterministic hash used for proof verification. Cannot be manipulated without valid proof.

- **What happens if it reverts, reenters or does other unusual control flow?**  
Pure function, no revert.
- `desertVerifier.Verify(proof, inputs)`
  - **What is controllable?** proof by user, and inputs is derived from parameters.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Returns boolean. If false, it causes a revert. The user cannot forge a valid proof without actual balance.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
External call but protected by `nonReentrant`. Could revert if proof is invalid.
- `increaseBalanceToWithdraw(_masterAccountIndex, _assetIndex, _totalBaseAmount)`
  - **What is controllable?** All parameters (but validated by proof).
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value. Updates withdrawal balance.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Internal function — could overflow if the amount is too large.

**Function:** `cancelOutstandingDepositsForDesertMode(uint64 _n, bytes[] memory _priorityPubData)`

This function allows cancellation of outstanding priority requests during desert mode. It verifies the hash chain and credits deposit balances back to users for deposit operations.

## Inputs

- `_n`
  - **Control:** Caller control.
  - **Constraints:** Must be greater than zero,  $\leq$  `openPriorityRequestCount`, and equal to `_priorityPubData.length`.
  - **Impact:** Number of requests to cancel.
- `_priorityPubData`
  - **Control:** Caller control (but must match the stored hash chain).
  - **Constraints:** Each element must be greater than zero and  $\leq$  `MAX_PRIORITY_REQUEST_PUBDATA_SIZE`. Must match stored priority-request hashes. Deposit operations must be the correct size.
  - **Impact:** Array of priority-request data to verify and cancel.

## Branches and code coverage

### Intended branches

- Desert mode must be active with outstanding requests to cancel.
  - ☒ Test coverage: ZkLighter.t.sol::L-461, L-464
- Hash chain should be verified and deposit balances credited via increaseBalanceToWithdraw.
  - ☒ Test coverage: ZkLighter.t.sol::L-498 to L-517
- Nondeposit operations should be skipped.
  - ☐ Test coverage: not tested

#### Negative behavior

- Revert when desert inactive, no outstanding deposits, or hash mismatch.
  - ☒ Negative test: ZkLighter.t.sol::L-461 to L-509 (partial coverage)
- Revert when `_n != _priorityPubData.length`.
  - ☐ Negative test: not tested

#### Function call analysis

- TxTypes.readDepositForDesertMode(depositPubdata)
  - **What is controllable?** depositPubdata (but must match the hash chain).
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns (accountIndex, assetIndex, baseAmount). Used to credit balance. Cannot be manipulated due to hash verification.
  - **What happens if it reverts, reenters or does other unusual control flow?** Library function, could revert if malformed.
- increaseBalanceToWithdraw(accountIndex, assetIndex, baseAmount)
  - **What is controllable?** Parameters from verified deposit data.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value. Updates balance.
  - **What happens if it reverts, reenters or does other unusual control flow?** Could overflow if the amount is too large.

#### Function: commitBatch(CommitBatchInfo calldata newBatchData, StoredBatchInfo calldata lastStoredBatch)

This is a validator-only function that commits a new batch of L2 blocks. It validates batch parameters, processes blob commitments (EIP-4844), updates state roots, and manages priority requests and on-chain operation queues.

## Inputs

- `newBatchData`
  - **Control:** Validator control.
  - **Constraints:** `pubdataCommitments` must be nonempty with Blob mode, `endBlockNumber` must increase, and `batchSize` must match block count. Timestamps must be nondecreasing, `priorityRequestCount` must not exceed available requests, and `prefixPriorityRequestHash` must match if requests exist.
  - **Impact:** New batch data to commit including block range, timestamps, state roots, priority requests, and blob commitments.
- `lastStoredBatch`
  - **Control:** Validator control (but must match stored hash).
  - **Constraints:** Must hash to `storedBatchHashes[committedBatchesCount]`.
  - **Impact:** Previous batch information for validation continuity.

## Branches and code coverage

### Intended branches

- Should validate batch parameters (block numbers, timestamps, batch size) and process blob commitments.
  - ☒ Test coverage: `ZkLighter.t.sol:L-214 to L-224`
- Should verify `lastStoredBatch` hash and priority-request hash chain.
  - ☒ Test coverage: `ZkLighter.t.sol:L-316, L-406`
- Should update counters and on-chain execution queue.
  - ☒ Test coverage: `ZkLighter.t.sol:L-329 to L-333`
- State-root override via `stateRootUpdates` mapping.
  - ☐ Test coverage: not tested

### Negative behavior

- Revert when caller is not validator or batch parameters invalid.
  - ☒ Negative test: `ZkLighter.t.sol:L-336 to L-416`
- Revert from `_processBlobs` if blob validation fails.
  - ☐ Negative test (not explicitly tested)

## Function call analysis

- `governance.isActiveValidator(msg.sender)`

- **What is controllable?** `msg.sender` by the caller.
- **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if not validator.
- **What happens if it reverts, reenters or does other unusual control flow?**  
Prevents unauthorized commits.
- `hashStoredBatchInfo(lastStoredBatch)`
  - **What is controllable?** `lastStoredBatch` by the validator.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Must match stored hash. Deterministic hash.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Pure function, no revert.
- `_processBlobs(newBatchData.pubdataCommitments[1:])`
  - **What is controllable?** Blob commitments by the validator.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Returns aggregated blob commitment. Validates blob proofs via precompile.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Could revert if blob validation fails. View function, no reentrancy.

### Function: `verifyBatch(StoredBatchInfo memory batch, bytes calldata proof)`

This is a validator-only function that verifies a committed batch using a ZK proof. It updates verification counters, state roots, and lazily updates execution state.

#### Inputs

- `batch`
  - **Control:** Validator control (but must match committed batch).
  - **Constraints:** `batchNumber` must be `verifiedBatchesCount + 1`. Must hash to stored batch hash.
  - **Impact:** Batch to verify.
- `proof`
  - **Control:** Validator control.
  - **Constraints:** Must be valid ZK proof for batch commitment.
  - **Impact:** Zero-knowledge proof of batch validity.

#### Branches and code coverage

##### Intended branches

- Should verify batches in order with valid ZK proof.
  - ☑ Test coverage: `ZkLighter.t.sol::L-442 to L-444`
- Should update verification counters and lazily execute if no pending on-chain batches.
  - ☑ Test coverage: `ZkLighter.t.sol::L-439, L-445, L-557 to L-614`

#### Negative behavior

- Revert when caller is not validator, batch not committed, or proof fails.
  - ☑ Negative test: `ZkLighter.t.sol::L-448, L-561, L-566`

#### Function call analysis

- `governance.isActiveValidator(msg.sender)`
  - **What is controllable?** `msg.sender` by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if not the validator.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Prevents unauthorized verification.
- `hashStoredBatchInfo(batch)`
  - **What is controllable?** `batch` by the validator.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Must match stored hash. Deterministic.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Pure function, no revert.
- `verifier.Verify(proof, inputs)`
  - **What is controllable?** `proof` by the validator and `inputs` from the batch commitment.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Returns boolean. If false, reverts. Validator cannot forge a valid proof.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
External call but protected by `nonReentrant`.

#### Function: `_executeOneBatch(StoredBatchInfo memory batch, bytes memory _onChainOperationsPubData)`

This is an internal function that processes on-chain operations (withdrawals) for a single batch. It decodes operation types, increases withdrawal balances, and validates operation's hash.

## Inputs

- batch
  - **Control:** Called by executeBatches with a validated batch.
  - **Constraints:** Must match the stored batch hash.
  - **Impact:** Batch containing the operations hash to verify.
- \_onChainOperationsPubData
  - **Control:** Called by executeBatches with caller-provided data.
  - **Constraints:** Must decode correctly and hash to batch.onChainOperationsHash.
  - **Impact:** Encoded on-chain operations to process.

## Branches and code coverage

### Intended branches

- Should verify batch hash and process operations sequentially.
  - ☒ Test coverage: ZkLighter.t.sol::L-574, L-777 to L-842
- Should decode and credit withdrawal balances via increaseBalanceToWithdraw.
  - ☒ Test coverage: ZkLighter.t.sol::L-628
- USDCWithdraw operations.
  - ☐ Test coverage: not tested

### Negative behavior

- Revert when batch hash mismatches, invalid operation type, or operations hash mismatches.
  - ☒ Negative test: ZkLighter.t.sol::L-917, L-963, L-1024, L-1087

## Function call analysis

- hashStoredBatchInfo(batch)
  - **What is controllable?** batch from caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Must match stored hash. Deterministic.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure function, no revert.
- TxTypes.readWithdrawOnChainLog(...) / TxTypes.readUSDCWithdrawOnChainLog(...)
  - **What is controllable?** Pubdata from caller, offset.

- **If the return value is controllable, how is it used and how can it go wrong?**  
Returns decoded withdrawal data. Must match hash to be valid.
- **What happens if it reverts, reenters or does other unusual control flow?**  
Could revert if malformed. Library functions.
- `increaseBalanceToWithdraw(_tx.masterAccountIndex, _tx.assetIndex, _tx.baseAmount)`
  - **What is controllable?** All parameters from decoded data (validated by hash).
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value. Updates balances.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Could overflow if the amount is too large.

### Function: `executeBatches(StoredBatchInfo[] memory batches, bytes[] memory onChainOperationsPubData)`

This function executes verified batches that contain on-chain operations (withdrawals). It processes withdrawal operations, updates state roots, and manages execution queues. It can only execute batches in the on-chain execution queue.

#### Inputs

- `batches`
  - **Control:** Caller control (but must be verified and in queue).
  - **Constraints:** Length must match `onChainOperationsPubData`. Length must be  $\leq$  `pendingOnChainBatchesCount`. Each batch must be verified and match queue order.
  - **Impact:** Array of batches to execute.
- `onChainOperationsPubData`
  - **Control:** Caller control (but must match batch operation's hash).
  - **Constraints:** Each element's operations must hash to the corresponding batch's `onChainOperationsHash`.
  - **Impact:** On-chain operation data for each batch.

#### Branches and code coverage

##### Intended branches

- Anyone can call (not restricted to validators). Array lengths must match.
- ☑ Test coverage: `ZkLighter.t.sol::L-723, L-773, L-853`
- Should process verified batches in queue order and update counters/state roots.

- ☑ Test coverage: `ZkLighter.t.sol::L-612 to L-645`
- Should support multiple operations per batch via `_executeOneBatch`.
- ☑ Test coverage: `ZkLighter.t.sol::L-777, L-842`

### Negative behavior

- Revert when array mismatch, batch not verified, or not in queue.
- ☑ Negative test: `ZkLighter.t.sol::L-853, L-861, L-871, L-1151`
- Revert from `_executeOneBatch` on validation errors.
- ☑ Negative test: `ZkLighter.t.sol::L-917, L-963, L-1024, L-1087`

### Function call analysis

- `_executeOneBatch(batches[i], onChainOperationsPubData[i])`
  - **What is controllable?** Both parameters by the caller (but validated).
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value. Processes withdrawals and validates hash.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Internal function — can revert if there is a hash mismatch or there are invalid operations.

**Function:** `revertBatches(StoredBatchInfo[] memory _batchesToRevert, StoredBatchInfo memory _remainingBatch)`

This is a validator-only function that reverts committed batches (rollback). It cannot revert executed batches. It updates all batch counters and verified state.

### Inputs

- `_batchesToRevert`
  - **Control:** Validator control (but must match committed batches).
  - **Constraints:** Each batch must match stored hash and be at `committedBatchesCount`. It cannot revert the genesis batch (`endBlockNumber != 0`). Must be pending (not executed). If it has on-chain ops, must be at the end of the pending queue.
  - **Impact:** Array of batches to revert from most recent backwards.
- `_remainingBatch`
  - **Control:** Validator control (but must match stored batch).
  - **Constraints:** Must hash to `storedBatchHashes[committedBatchesCount]` after reversions. Must be the last remaining batch.

- **Impact:** The batch that remains after reversions (for validation).

## Branches and code coverage

### Intended branches

- Caller must be an active validator. Should allow no-op rollback.
  - ☒ Test coverage: `Proxy.test.ts:L-148` (no-op case only)
- Should revert committed batches and update counters.
  - ☐ Test coverage: not tested (actual batch reversion)

### Negative behavior

- Revert when not validator, genesis batch, executed batch, or hash mismatch.
  - ☐ Negative test: not tested

## Function call analysis

- `governance.isActiveValidator(msg.sender)`
  - **What is controllable?** `msg.sender` by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if not a validator.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Prevents unauthorized reversions.
- `hashStoredBatchInfo(storedBatchInfo) / hashStoredBatchInfo(_remainingBatch)`
  - **What is controllable?** Batch data by the validator.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Must match stored hash. Deterministic.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Pure functions, no revert.

### Function: `transferERC20(IERC20 _token, address _to, uint256 _amount, uint256 _maxAmount)`

This is a self-callable function that transfers ERC-20 tokens and returns the actual transferred amount. It handles fee-on-transfer tokens by checking the balance difference. It is used by `withdrawPendingBalance` for isolated state changes.

## Inputs

- `_token`
  - **Control:** Caller control (but must be self-call).
  - **Constraints:** Must be called from `address(this)`.
  - **Impact:** ERC-20 token to transfer.
- `_to`
  - **Control:** Caller control (but must be self-call).
  - **Constraints:** Recipient address.
  - **Impact:** Transfer recipient.
- `_amount`
  - **Control:** Caller control (but must be self-call).
  - **Constraints:** Amount to transfer.
  - **Impact:** Requested transfer amount.
- `_maxAmount`
  - **Control:** Caller control (but must be self-call).
  - **Constraints:** Maximum allowed balance decrease.
  - **Impact:** Upper bound for balance decrease (protects against fee-on-transfer issues).

## Branches and code coverage

### Intended branches

- Should only be callable as self-call and return actual balance difference.
  - ☒ Test coverage: implicitly via `withdrawPendingBalance` tests

### Negative behavior

- Revert when `msg.sender != address(this)` (`ZkLighter_OnlyZkLighter`).
  - ☐ Negative test (self-call pattern not explicitly tested externally)
- Revert when `balance diff > maxAmount` (`ZkLighter_RollUpBalanceBiggerThanMaxAmount`).
  - ☐ Negative test (would require malicious token)
- Revert when `safeTransfer` fails (from `SafeERC20`).
  - ☐ Negative test (not simulated in tests)

## Function call analysis

- `_token.balanceOf(address(this))`
  - **What is controllable?** Token address (validated by the caller).
  - **If the return value is controllable, how is it used and how can it go wrong?** Used to calculate the actual transferred amount. For fee-on-transfer tokens, may differ from requested.
  - **What happens if it reverts, reenters or does other unusual control flow?** External view call. Token could be malicious but validated by system.
- `SafeERC20.safeTransfer(_token, _to, _amount)`
  - **What is controllable?** All parameters from self-call.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value — reverts on failure.
  - **What happens if it reverts, reenters or does other unusual control flow?** External call. Reentrancy protected by the self-call pattern and `nonReentrant` in the caller.

## Function: `transferETH(address _to, uint256 _amount)`

This is a self-callable function that transfers ETH to a recipient. It is used by `withdrawPendingBalance` for isolated state changes that can be reverted if needed.

## Inputs

- `_to`
  - **Control:** Caller control (but must be self-call).
  - **Constraints:** Must be called from `address(this)`.
  - **Impact:** ETH recipient.
- `_amount`
  - **Control:** Caller control (but must be self-call).
  - **Constraints:** Amount of ETH to transfer.
  - **Impact:** ETH amount to send.

## Branches and code coverage

### Intended branches

- Should only be callable as self-call and transfer ETH to recipient.
  - ☒ Test coverage: `ZkLighter.t.sol::L-706` (via `withdrawPendingBalance`)

### Negative behavior

- Revert when not self-call or ETH transfer fails.
- Negative test: not tested

### Function call analysis

- `_to.call{value: _amount}("")`
  - **What is controllable?** Recipient and amount from self-call.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns bool success. Must be true or revert.
  - **What happens if it reverts, reenters or does other unusual control flow?** Low-level call. Reentrancy protected by the self-call pattern and `nonReentrant` in the caller. Recipient could be a contract with fallback.

### Function: `increaseBalanceToWithdraw(uint48 _masterAccountIndex, uint16 _assetIndex, uint128 _baseAmount)`

This is an internal function that increases the withdrawable balance for an account/asset pair. It is used when L2 withdrawals are executed, deposits are canceled in desert mode, or desert-mode exits are performed.

### Inputs

- `_masterAccountIndex`
  - **Control:** Called by various functions (`performDesert`, `cancelOutstandingDepositsForDesertMode`, `_executeOneBatch`).
  - **Constraints:** Should be a valid master-account index.
  - **Impact:** The master account whose withdrawable balance increases.
- `_assetIndex`
  - **Control:** From calling function — should be validated.
  - **Constraints:** Should be a valid asset index.
  - **Impact:** The asset whose balance increases.
- `_baseAmount`
  - **Control:** From calling function (validated or proven).
  - **Constraints:** Must fit in `uint128`. Added to existing balance (overflow risk).
  - **Impact:** Amount to add to withdrawable balance in base units.

## Branches and code coverage

### Intended branches

- Should increase pending balance for the account/asset pair.
- ☒ Test coverage: `ZkLighter.t.sol::L-458, L-517, L-628, L-636`

### Negative behavior

- Overflow when adding balances (`uint128` limit).
- ☐ Negative test (would require depositing to be greater than the `uint128` maximum, implicit Solidity 0.8.x protection)

## Function call analysis

No external or internal function calls (direct storage update).

### Function: `getPendingBalance(address _owner, uint16 _assetIndex)`

This is an external view function that returns the pending withdrawable balance for an owner address and a specific asset. It maps the address to the master-account index and looks up the balance.

### Inputs

- `_owner`
  - **Control:** Caller control.
  - **Constraints:** Any address.
  - **Impact:** Address to check the balance for.
- `_assetIndex`
  - **Control:** Caller control.
  - **Constraints:** Any `uint16` (no validation).
  - **Impact:** Asset to check the balance for.

## Branches and code coverage

### Intended branches

- Should return pending balance for owner/asset pair (0 for unregistered).
- ☒ Test coverage: `ZkLighter.t.sol::L-628, L-636, L-703, L-711`

### Negative behavior

- View function, no revert conditions.

### Function call analysis

- `getAccountIndexFromAddress(_owner)`
  - **What is controllable?** `_owner` by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns stored mapping. If not registered, returns `NIL_ACCOUNT_INDEX`, which will have 0 balance.
  - **What happens if it reverts, reenters or does other unusual control flow?** View function, no unusual flow.

### Function: `getPendingBalanceLegacy(address _owner)`

This is a public view function that returns the pending USDC balance from the deprecated/legacy balance storage. It is used for backwards compatibility with older deposits before multiasset support.

### Inputs

- `_owner`
  - **Control:** Caller control.
  - **Constraints:** Any address.
  - **Impact:** Address to check the legacy balance for.

### Branches and code coverage

#### Intended branches

- Should return legacy USDC balance from deprecated storage.

☐ Test coverage: not tested

### Negative behavior

- View function, no revert conditions.

### Function call analysis

- `getAccountIndexFromAddress(_owner)`

- **What is controllable?** `_owner` by the caller.
- **If the return value is controllable, how is it used and how can it go wrong?**  
Returns stored mapping. If not registered, returns `NIL_ACCOUNT_INDEX`, which will have 0 balance.
- **What happens if it reverts, reenters or does other unusual control flow?**  
View function, no unusual flow.

**Function:** `withdrawPendingBalance(address _owner, uint16 _assetIndex, uint128 _baseAmount)`

This function allows users to withdraw their pending L1 balances accumulated from L2 withdrawals or desert-mode exits. It handles both ETH and ERC-20 tokens, with support for fee-on-transfer tokens.

## Inputs

- `_owner`
  - **Control:** Caller control.
  - **Constraints:** Must have a registered master account. Must have sufficient pending balance.
  - **Impact:** Owner whose balance to withdraw.
- `_assetIndex`
  - **Control:** Caller control.
  - **Constraints:** Must be valid asset (`NATIVE_ASSET_INDEX` or configured).
  - **Impact:** Asset to withdraw.
- `_baseAmount`
  - **Control:** Caller control.
  - **Constraints:** Must be greater than zero and less than or equal to the pending balance.
  - **Impact:** Amount to withdraw in base units.

## Branches and code coverage

### Intended branches

- Should withdraw pending balance for registered owner (ETH or ERC-20).
- ☑ Test coverage: `ZkLighter.t.sol::L-631, L-638, L-706, L-713`

### Negative behavior

- Revert when amount exceeds balance, is zero, asset invalid, or transfer fails.

❑ Negative test: not tested

## Function call analysis

- `getAccountIndexFromAddress(_owner)`
  - **What is controllable?** `_owner` by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Returns master account from mapping. Cannot be manipulated.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
View function, no unusual flow.
- `this.transferETH(_owner, amount)`
  - **What is controllable?** `_owner` and `amount` (validated).
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Returns transferred amount. Should equal requested for ETH.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
External self-call to isolate state changes. Can revert if transfer fails. Protected by `nonReentrant`.
- `this.transferERC20(IERC20(assetConfig.tokenAddress), _owner, amount, balance)`
  - **What is controllable?** Recipient and amounts (validated).
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Returns the actual transferred amount. May differ from that requested for fee-on-transfer tokens. Cannot exceed balance.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
External self-call. Can revert if transfer fails. Protected by `nonReentrant`.
- `SafeCast.toUint128(transferredBaseAmount)`
  - **What is controllable?** Calculated transferred amount.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Must fit in `uint128`. Could overflow for very large amounts.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Reverts on overflow.

**Function:** `withdrawPendingBalanceLegacy(address _owner, uint128 _baseAmount)`

This is an external function to withdraw the pending USDC balance from the deprecated/legacy balance storage. It allows users to withdraw balances accumulated before multiasset support was added.

## Inputs

- `_owner`
  - **Control:** Caller control.
  - **Constraints:** Must have a registered master account. Must have sufficient legacy pending balance.
  - **Impact:** Owner whose legacy balance to withdraw.
- `_baseAmount`
  - **Control:** Caller control.
  - **Constraints:** Must be greater than zero and less than or equal to the legacy pending balance.
  - **Impact:** Amount to withdraw in base units (USDC has 1:1 base unit).

## Branches and code coverage

### Intended branches

- Should withdraw legacy USDC balance from deprecated storage.
- ☐ Test coverage: not tested

### Negative behavior

- Revert when amount exceeds balance, is zero, or transfer fails.
- ☐ Negative test: not tested

## Function call analysis

- `getAccountIndexFromAddress(_owner)`
  - **What is controllable?** `_owner` by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns master account from mapping. Cannot be manipulated.
  - **What happens if it reverts, reenters or does other unusual control flow?** View function, no unusual flow.
- `governance.usdc()`
  - **What is controllable?** Nothing — reads USDC address from governance.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns USDC token contract address. Set during initialization.
  - **What happens if it reverts, reenters or does other unusual control flow?** View function — could revert if governance is not set.
- `this.transferERC20(governance.usdc(), _owner, uint256(_baseAmount), uint256(baseBalance))`

- **What is controllable?** Recipient and amounts (validated).
- **If the return value is controllable, how is it used and how can it go wrong?** Returns actual transferred amount. Should equal requested for standard USDC.
- **What happens if it reverts, reenters or does other unusual control flow?** External self-call. Can revert if transfer fails. Protected by nonReentrant.

### Function: activateDesertMode()

This is a public function that anyone can call to activate desert mode if priority requests have expired. Once activated, users can exit using performDesert and cancel outstanding deposits.

### Branches and code coverage

#### Intended branches

- Trigger should be true if `openPriorityRequestCount > 0` *and* the current time is greater than or equal to the expiration timestamp *and* `expiration != 0`.
  - ☒ Test coverage: `test/ZkLighter.test.ts::L-1475` (sets up priority requests and advances time to activate)
- If the trigger is true and not already in desert mode, activate it.
  - ☒ Test coverage: `test/ZkLighter.test.ts::L-1488` (calls `activateDesertMode()`), `test/ZkLighter.test.ts::L-1491` (desertMode becomes true)
- If already in desert mode, do not emit the event again.
  - ☐ Test coverage (idempotency not explicitly tested)
- The DesertMode event should be emitted on activation.
  - ☒ Test coverage: `test/ZkLighter.test.ts::L-1488`
- The function should return true if triggered and false otherwise.
  - ☐ Test coverage (return value not asserted)

#### Negative behavior

- No revert conditions (permissionless function by design).

### Function call analysis

No external or internal function calls (direct storage reads).

## Function: `setTreasury (address _newTreasury)`

This is a governor-only function to update the treasury address that collects fees. The treasury address must not be in use as a user account.

### Inputs

- `_newTreasury`
  - **Control:** Governor control.
  - **Constraints:** Cannot be `address(0)`. Cannot be registered as a user account (must return `NIL_ACCOUNT_INDEX`).
  - **Impact:** New treasury address.

### Branches and code coverage

#### Intended branches

- The new treasury must not be `address(0)`.
  - ☒ Test coverage: `test/ZkLighter.system.t.sol::L-45`
- The new treasury must not be registered as a user account.
  - ☒ Test coverage: `test/ZkLighter.system.t.sol::L-51`
- The treasury should be updated.
  - ☒ Test coverage: `test/ZkLighter.system.t.sol::L-30`
- The `TreasuryUpdate` event should be emitted.
  - ☒ Test coverage: `test/ZkLighter.system.t.sol::L-31`

#### Negative behavior

- Revert when the caller is not the governor.
  - ☒ Negative test: `test/ZkLighter.system.t.sol::L-39`
- Revert when the treasury is `address(0)` (`ZkLighter_TreasuryCannotBeZero`).
  - ☒ Negative test: `test/ZkLighter.system.t.sol::L-45`
- Revert when the treasury is a registered account (`ZkLighter_TreasuryCannotBeInUse`).
  - ☒ Negative test: `test/ZkLighter.system.t.sol::L-51`

### Function call analysis

- `governance.requireGovernor(msg.sender)`
  - **What is controllable?** `msg.sender` by the caller.

- **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if not the governor.
- **What happens if it reverts, reenters or does other unusual control flow?**  
Prevents unauthorized updates.
- `getAccountIndexFromAddress(_newTreasury)`
  - **What is controllable?** `_newTreasury` by the governor.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Must return `NIL_ACCOUNT_INDEX`. Cannot be manipulated.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
View function, no unusual flow.

### Function: `setInsuranceFundOperator(address _newInsuranceFundOperator)`

This is a governor-only function that updates the insurance-fund-operator address. The insurance-fund-operator address must not be in use as a user account.

#### Inputs

- `_newInsuranceFundOperator`
  - **Control:** Governor control.
  - **Constraints:** Cannot be address(0). Cannot be registered as a user account (must return `NIL_ACCOUNT_INDEX`).
  - **Impact:** New insurance-fund-operator address.

### Branches and code coverage

#### Intended branches

- The new operator must not be address(0).
  - ☒ Test coverage: `test/ZkLighter.system.t.sol::L-83`
- The new operator must not be registered as a user account.
  - ☒ Test coverage: `test/ZkLighter.system.t.sol::L-89`
- The insurance fund operator should be updated.
  - ☒ Test coverage: `test/ZkLighter.system.t.sol::L-68`
- The `InsuranceFundOperatorUpdate` event should be emitted.
  - ☒ Test coverage: `test/ZkLighter.system.t.sol::L-69`

#### Negative behavior

- Revert when the caller is not the governor.

- ☒ Negative test: `test/ZkLighter.system.t.sol::L-77`
- Revert when the operator is `address(0)` (`ZkLighter_InsuranceFundOperatorCannotBeZero`).
- ☒ Negative test: `test/ZkLighter.system.t.sol::L-83`
- Revert when the operator is a registered account (`ZkLighter_InsuranceFundOperatorCannotBeInUse`).
- ☒ Negative test: `test/ZkLighter.system.t.sol::L-89`

## Function call analysis

- `governance.requireGovernor(msg.sender)`
  - **What is controllable?** `msg.sender` by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value — reverts if not the governor.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Prevents unauthorized updates.
- `getAccountIndexFromAddress(_newInsuranceFundOperator)`
  - **What is controllable?** `_newInsuranceFundOperator` by the governor.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Must return `NIL_ACCOUNT_INDEX`. Cannot be manipulated.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
View function, no unusual flow.

## Function: `delegateAdditional()`

Internal function that delegates calls to `AdditionalZkLighter` using low-level `delegatecall`. Used by multiple external functions to split contract logic due to size limits. Uses `msg.data` directly.

## Inputs

None (uses `msg.data` directly from original call).

## Branches and code coverage

### Intended branches

- Should `delegatecall` to `AdditionalZkLighter` with full calldata.
- ☒ Test coverage: all delegated functions work correctly
- Should copy return data back to caller.

- ☒ Test coverage: return values propagate correctly
- If `delegatecall` fails, should revert with return data.
- ☒ Test coverage: error messages propagate correctly

### Negative behavior

- Revert when called on implementation contract (not via proxy).
- ☐ Negative test: not tested
- Revert with `delegatecall` revert data if target reverts.
- ☒ Negative test: all negative tests on delegated functions verify error propagation

### Function call analysis

- Assembly `delegatecall`
  - **What is controllable?** Calldata from original caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Return data passed through unchanged. Preserves `msg.sender` and `msg.value` context.
  - **What happens if it reverts, reenters or does other unusual control flow?** Reverts propagate correctly. Reentrancy protection must be in external caller or `AdditionalZkLighter` functions (all use `nonReentrant`).

### Function: `_pointEvaluationPrecompile(bytes32 blobVersionedHash, bytes calldata evaluationPointCommitmentProof)`

This is an internal view function that calls the EIP-4844 point evaluation precompile to verify blob KZG proofs. It validates that the precompile succeeded and returned the expected `BLS_MODULUS` value.

### Inputs

- `blobVersionedHash`
  - **Control:** From blobhash opcode (transaction-provided).
  - **Constraints:** Must be a valid versioned hash from transaction blobs.
  - **Impact:** Identifies which blob to verify.
- `evaluationPointCommitmentProof`
  - **Control:** From `pubDataCommitments` (validator-provided).
  - **Constraints:** Must be a valid KZG proof (160 bytes — `evaluationPoint` 64 bytes plus commitment 48 bytes plus proof 48 bytes).

- **Impact:** KZG proof data to verify.

## Branches and code coverage

### Intended branches

- Precompile input should be encoded correctly (hash plus proof).
  - ☒ Test coverage: exercised via blob processing in `test/ZkLighter.t.sol::L-215` (blobhashes set) and `test/ZkLighter.t.sol::L-222` (commitBatch happy path)
- Static call to `POINT_EVALUATION_PRECOMPILE_ADDRESS` should succeed.
  - ☒ Test coverage: exercised via blob processing in `test/ZkLighter.t.sol::L-215` (blobhashes set) and `test/ZkLighter.t.sol::L-222` (commitBatch happy path)
- Response should decode to two `uint256` values.
  - ☒ Test coverage: exercised via blob processing in `test/ZkLighter.t.sol::L-215` (blobhashes set) and `test/ZkLighter.t.sol::L-222` (commitBatch happy path)
- Second `uint256` (result) must equal `BLS_MODULUS`.
  - ☒ Test coverage: exercised via blob processing in `test/ZkLighter.t.sol::L-215` (blobhashes set) and `test/ZkLighter.t.sol::L-222` (commitBatch happy path)

### Negative behavior

- Revert when precompile call fails (`ZkLighter_InvalidPointEvaluationParams`).
  - ☐ Negative test (not explicitly tested)
- Revert when `result != BLS_MODULUS` (`ZkLighter_InvalidPointEvaluationParams`).
  - ☐ Negative test (not explicitly tested)

## Function call analysis

- `POINT_EVALUATION_PRECOMPILE_ADDRESS.staticcall(precompileInput)`
  - **What is controllable?** Precompile input (hash plus proof data).
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns (bool success, bytes data). Must succeed, and data must decode correctly with `result == BLS_MODULUS`.
  - **What happens if it reverts, reenters or does other unusual control flow?** Static call, no state changes possible. No reentrancy risk.

**Function: `_processBlobs(bytes calldata pubDataCommitments)`**

This is an internal view function that processes and validates EIP-4844 blob commitments. It verifies each blob using the point evaluation precompile and computes an aggregated commitment hash.

**Inputs**

- `pubDataCommitments`
  - **Control:** Called by `commitBatch` with validator-provided data.
  - **Constraints:** Length must be a multiple of `BLOB_DATA_COMMITMENT_BYTE_SIZE` (160 bytes). Blob count must be  $\leq$  `MAX_BLOB_COUNT`. Each blob must have a valid versioned hash. Must pass point-evaluation-precompile verification.
  - **Impact:** Array of blob commitment data (`evaluationPointX`, `evaluationPointY`, `commitment`, `proof`) per blob.

**Branches and code coverage****Intended branches**

- The `pubDataCommitments` length must be a multiple of 160 bytes.
  - ☑ Test coverage: exercised via `test/ZkLighter.t.sol::L-222` (commit batch with blob mode + commitments)
- The blob count must be calculated correctly and  $\leq$  `MAX_BLOB_COUNT`.
  - ☑ Test coverage: exercised with a single blob via `test/ZkLighter.t.sol::L-215` and `test/ZkLighter.t.sol::L-222`
- For each blob, extract `evaluationPointX` and `evaluationPointY`.
  - ☑ Test coverage: exercised (not explicitly asserted) via `test/ZkLighter.t.sol::L-222`
- For each blob, get the versioned hash from the `blobhash(i)` opcode.
  - ☑ Test coverage: exercised via `test/ZkLighter.t.sol::L-215` and `test/ZkLighter.t.sol::L-222`
- The blob versioned hash must be nonzero.
  - ☑ Test coverage: exercised implicitly (call succeeds with configured blobhash) via `test/ZkLighter.t.sol::L-215` and `test/ZkLighter.t.sol::L-222`
- The point evaluation precompile should be called and succeed for each blob.
  - ☑ Test coverage: exercised via `test/ZkLighter.t.sol::L-222` (commit batch succeeds, so precompile verification does not revert)
- The current blob commitment should be a hash of (`evaluationPointX`,

```
evaluationPointY, blobVersionedHash).
```

- ☒ Test coverage: exercised (not explicitly asserted) via `test/ZkLighter.t.sol::L-222`
- The first blob sets `aggregatedBlobCommitment` and subsequent blobs hash with previous.
  - ☒ Test coverage: exercised for the first-blob case via `test/ZkLighter.t.sol::L-222`
- After processing all blobs, the next blobhash must be zero (no extra blobs).
  - ☒ Test coverage: exercised implicitly (call succeeds with exactly one configured blobhash) via `test/ZkLighter.t.sol::L-215` and `test/ZkLighter.t.sol::L-222`
- Return aggregated blob commitment.
  - ☒ Test coverage: exercised (return value is used by `commitBatch`, but not asserted directly) via `test/ZkLighter.t.sol::L-222`

### Negative behavior

- Revert when the `pubDataCommitments` length is not a multiple of 160 (`ZkLighter_InvalidBlobCommitmentParams`).
  - ☐ Negative test: not explicitly tested (would require `commitBatch` with `newBatchData.pubdataCommitments[1:]` not a multiple of 160 bytes)
- Revert when blob count is `> MAX_BLOB_COUNT` (`ZkLighter_InvalidBlobCount`).
  - ☐ Negative test: not explicitly tested (would require `pubdataCommitments` containing `MAX_BLOB_COUNT + 1` blob entries)
- Revert when `blobVersionedHash` is `bytes32(0)` (`ZkLighter_InvalidBlobCommitmentParams`).
  - ☐ Negative test: not explicitly tested (would require `vm.blobhashes` with an empty list, or otherwise forcing `blobhash(0) == 0`)
- Revert from `_pointEvaluationPrecompile` if verification fails.
  - ☐ Negative test: not explicitly tested (no test forces the point-evaluation precompile to return a non-BLS\_MODULUS value)
- Revert when extra blobs are attached (`emptyBlobVersionedHash != 0`) (`ZkLighter_InvalidBlobCommitmentParams`).
  - ☐ Negative test: not explicitly tested (would require setting `vm.blobhashes` with more blobs than the provided commitments cover)

### Function call analysis

- `_pointEvaluationPrecompile(blobVersionedHash, _blobDataCommitment)`

- **What is controllable?** Blob commitment data by validator and the versioned hash from the transaction.
- **If the return value is controllable, how is it used and how can it go wrong?** No return value — reverts if invalid. Uses EIP-4844 precompile to verify KZG proofs.
- **What happens if it reverts, reenters or does other unusual control flow?** Reverts if precompile fails or the result is invalid. Static call, no reentrancy.

## 6. Assessment Results

During our assessment on the scoped Lighter (EVM) contracts, we discovered four findings. No critical issues were found. Two findings were of medium impact, one was of low impact, and the remaining finding was informational in nature.

The Lighter EVM codebase demonstrates solid architectural design with appropriate security mechanisms. The code quality is generally high with clear structure.

To note, our audit was conducted under the assumption that off-chain components (L2 operator, sequencer, and ZK proof generation) perform their own validation and operate correctly. The security of the overall system is heavily dependent on these off-chain components, which handle critical validation logic for batch data integrity, L2 state transitions, and priority-request processing.

Additionally, the test suite has notable gaps in coverage for EIP-4844 blob processing, batch reversion logic, proxy upgrade mechanisms, and various edge cases. We recommend expanding test coverage in these areas before mainnet deployment to strengthen confidence in the system's security posture.

---

### 6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.